

Big Data Analytics



HDFS and MapReduce

Mihaela Breabăn

© FII 2025-2026

Acknowledgements: a consistent part of the presentation is based on Cloudera materials

Apache Hadoop

- ▶ A software framework for **storing** and **processing** large amounts of structured/**unstructured** data

- ▶ Features:

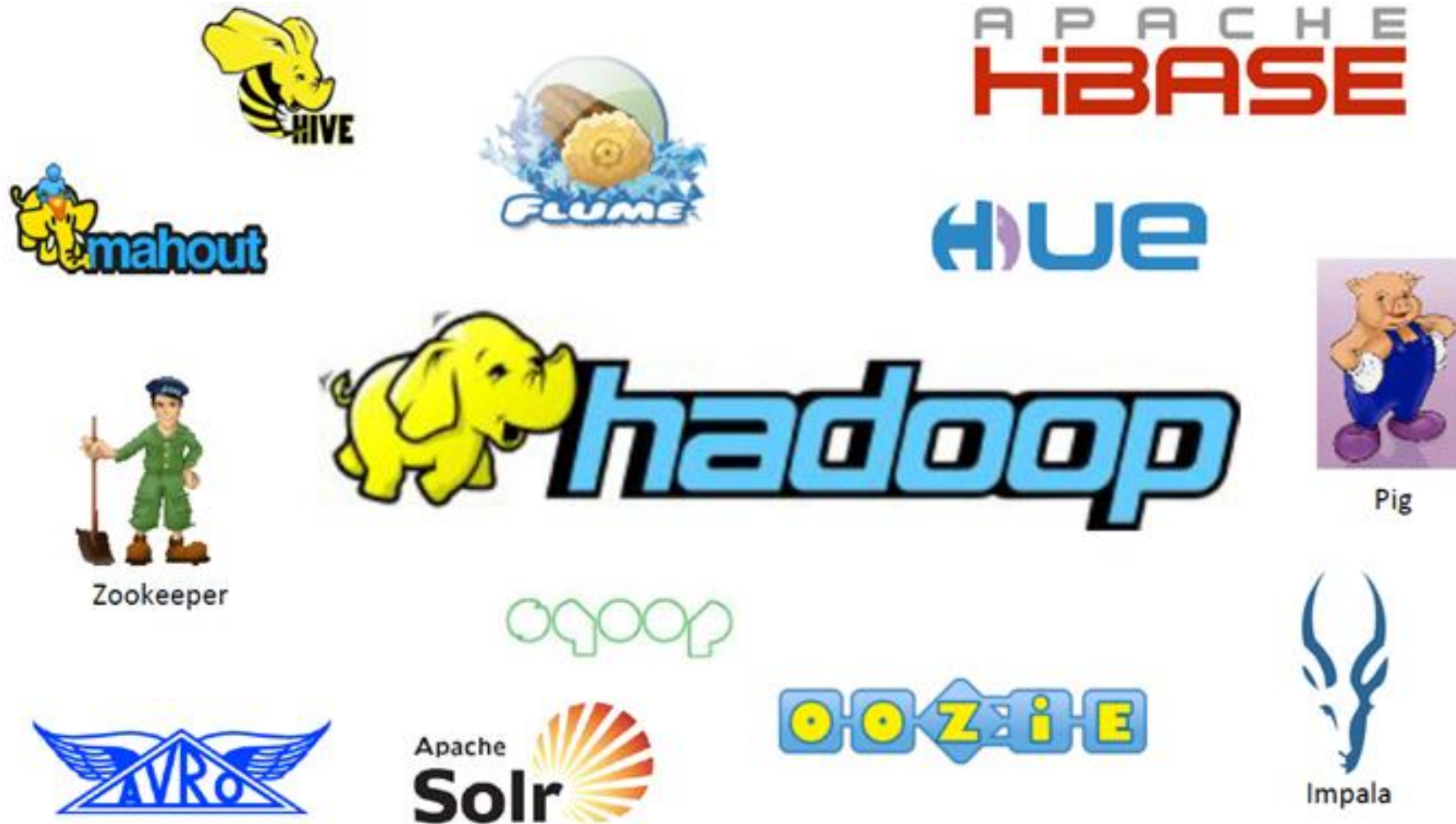
- ▶ Distributed
- ▶ Scalable
- ▶ Fault-tolerant
- ▶ Open source

username	name	organization	roles	timezone
aajisaka	Akira Ajsaka	NTT DATA		+9
acmurthy	Arun C. Murthy	Hortonworks		-8
amreshwari	Amareshwari Sriramadasu	InMobi		+5.5
arp	Arpit Agarwal	Hortonworks		-8
asuresh	Arun Suresh	Microsoft		-8
atm	Aaron T. Myers	Cloudera		-8
bikas	Bikas Saha	Hortonworks		-8
bobby	Robert(Bobby) Evans	Yahoo!		-6
brandonli	Brandon LI	Hortonworks		-8
cdouglas	Chris Douglas	Microsoft		-8
cmccabe	Colin Patrick McCabe	Cloudera	HDFS	-8
cnauroth	Chris Nauroth	Hortonworks		-8
cutting	Doug Cutting	Cloudera		-8
daryn	Daryn Sharp	Yahoo!		-6
ddas	Devaraj Das	Hortonworks		-8
devaraj	Devaraj K	Intel		+5.5
dhruba	Dhruba Borthakur	Facebook		-8
eli	Ell Collins	Cloudera		-8
enis	Enis Soztutar	Hortonworks		-8
epayne	Eric Payne	Yahoo		-6
gkesavan	Giridharan Kesavan	Hortonworks		-8
hairong	Hairong Kuang	Facebook		-8
hitesh	Hitesh Shah	Hortonworks		-8
jeagles	Jonathan Eagles	Yahoo!		-6
jghoman	Jakob Homan	LinkedIn		-8
jianhe	Jian He	Hortonworks		-9
jing9	Jing Zhao	Hortonworks		-8
jitendra	Jitendra Nath Pandey	Hortonworks		-8
jlowe	Jason Lowe	Yahoo!		-6
junping_du	Junping Du	Hortonworks		+8
kasha	Karthik Kambatla	Cloudera		-8
kihwal	Kihwal Lee	Yahoo!		-6
lei	Lei Xu	Cloudera		-8
liu	Luke Liu	VMware		-8
mahadev	Mahadev Konar	Hortonworks		-8
mattf	Matt Foley	Hortonworks		-8
mingma	Ming Ma	Twitter		-8
nigel	Nigel Daley	Jive		-8
omalley	Owen O'Malley	Hortonworks		-8
ozawa	Tsuyoshi Ozawa	NTT		+9
phunt	Patrick Hunt	Cloudera	ZooKeeper	-8
rangadi	Raghu Rangadi	Twitter		-8
rkanter	Robert Kanter	Cloudera		-8
rohithsharmaks	Rohith Sharma K S	Hortonworks		+5.5
sharad	Sharad Agarwal	InMobi		+5.5
shv	Konstantin Shvachko	LinkedIn	HDFS	-8
sandy	Sandy Ryza	Cloudera		-8
sjlee	Sangjin Lee	Twitter		-8
sradia	Sanjay Radia	Hortonworks		-8
sseth	Siddharth Seth	Hortonworks		-8
stack	Michael Stack	Cloudera	HBase	-8
stack	Michael Stack	Cloudera		-8

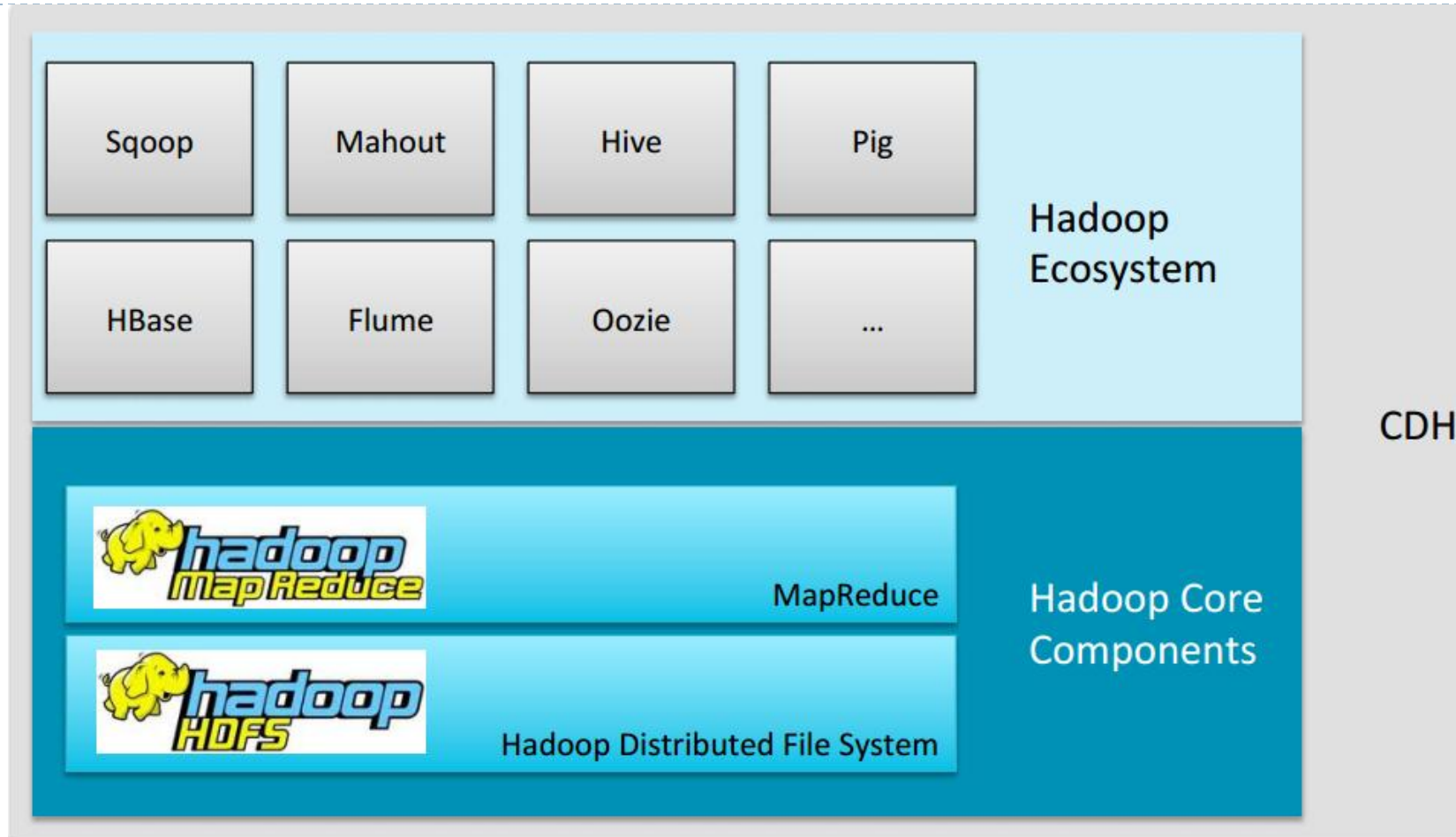
Hundreds of contributors:

<http://hadoop.apache.org/who.html>

A large ecosystem



The Hadoop Ecosystem

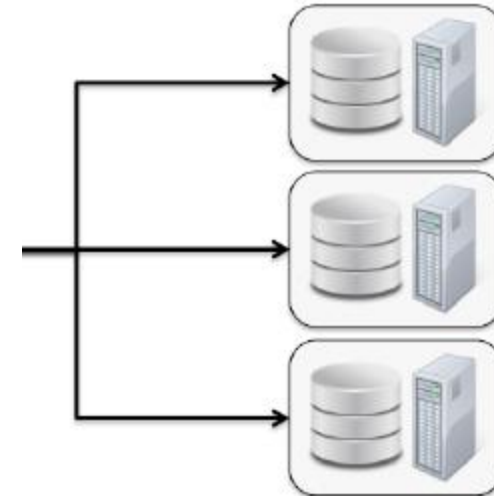


Apache Hadoop

Core Concepts



- ▶ Based on work done at Google
 - ▶ Google File System (2003) -> HDFS
 - ▶ MapReduce (2004) -> the most mature and accessible implementation
- ▶ Distribute the data and run computation where the data is stored
 - ▶ Nodes communicate less
 - ▶ Data is replicated for increased availability and for reliability
- ▶ Applications are written in high level code
 - ▶ No network programming

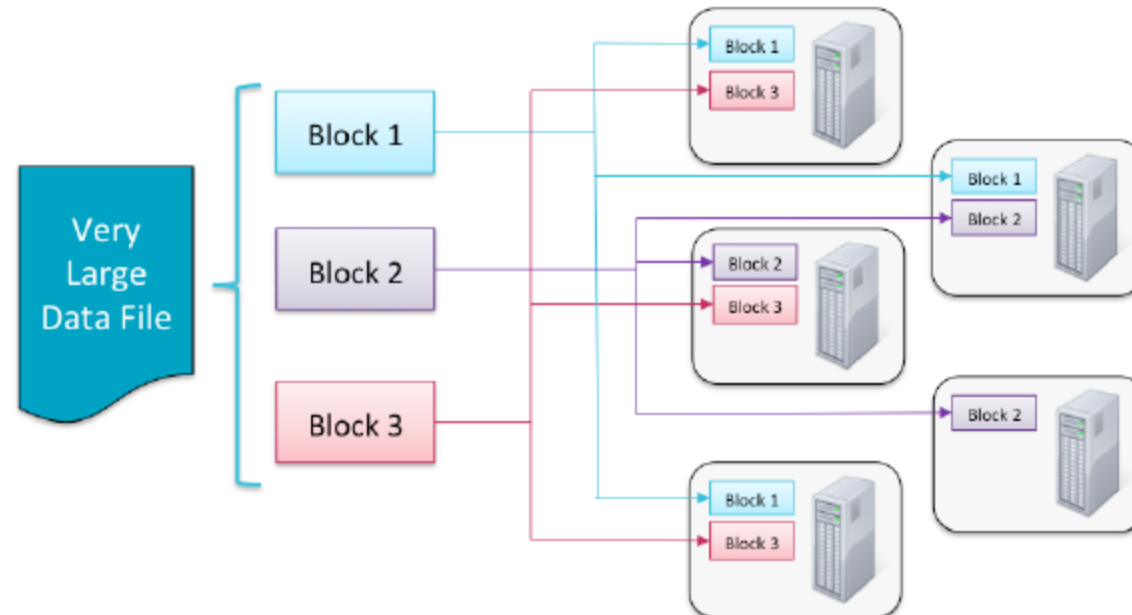
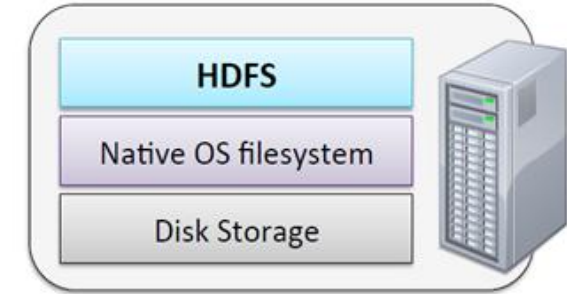


- ▶ A **Hadoop cluster**: a set of machines (commodity hardware) running HDFS and MapReduce

Hadoop Distributed Filesystem (HDFS)

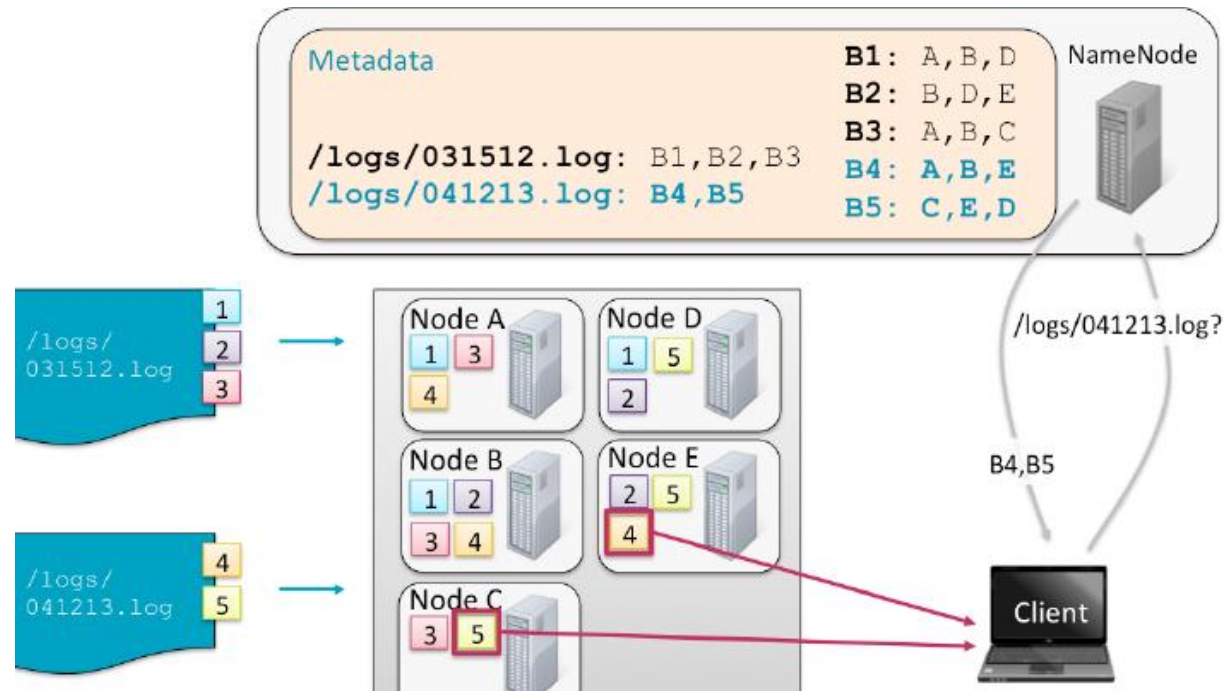
- ▶ Responsible for storing and retrieving data over the cluster
- ▶ Built over the native file system
- ▶ Write once / read many times
- ▶ Optimized for large streaming reads of files

Data is split into blocks/chunks of fixed size (typically 64 or 128 MB) at load time; each block is replicated on several nodes and is stored as a distinct file in a set of directories specified in configuration files

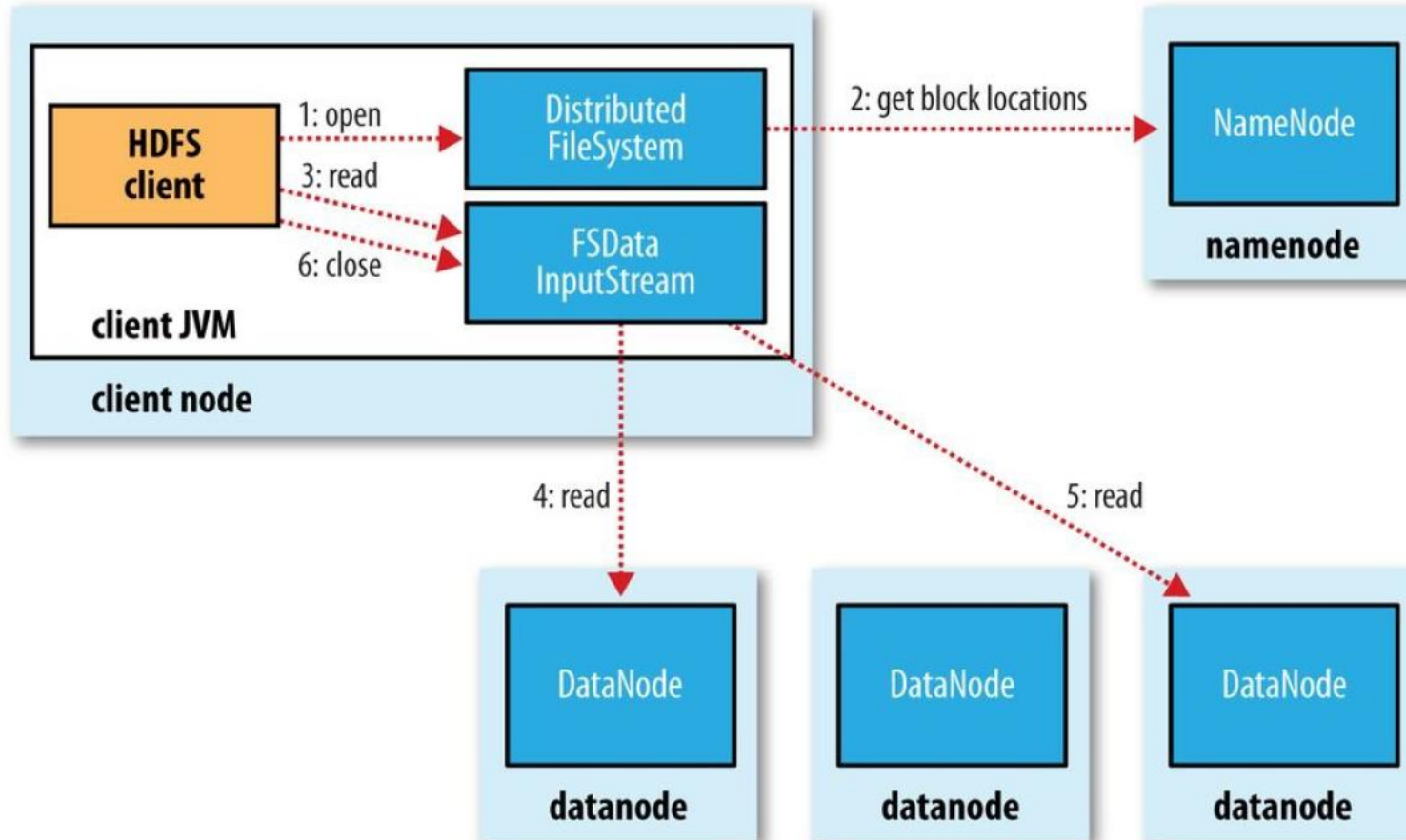


Reading data stored on HDFS

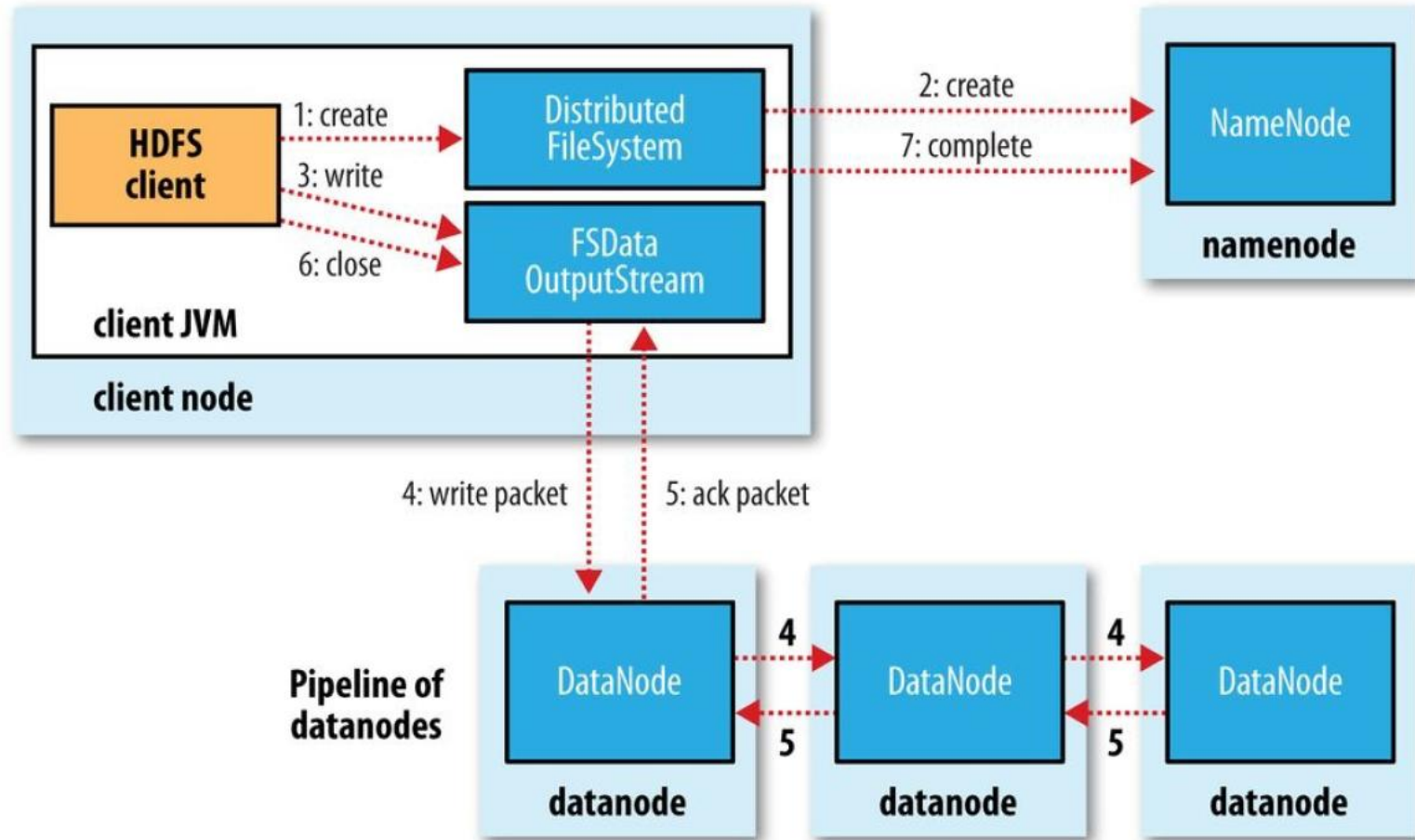
- ▶ NameNode – stores the metadata
- ▶ DataNode – stores the blocks of operational data



Reading data stored on HDFS



Writing data on HDFS



Accessing HDFS

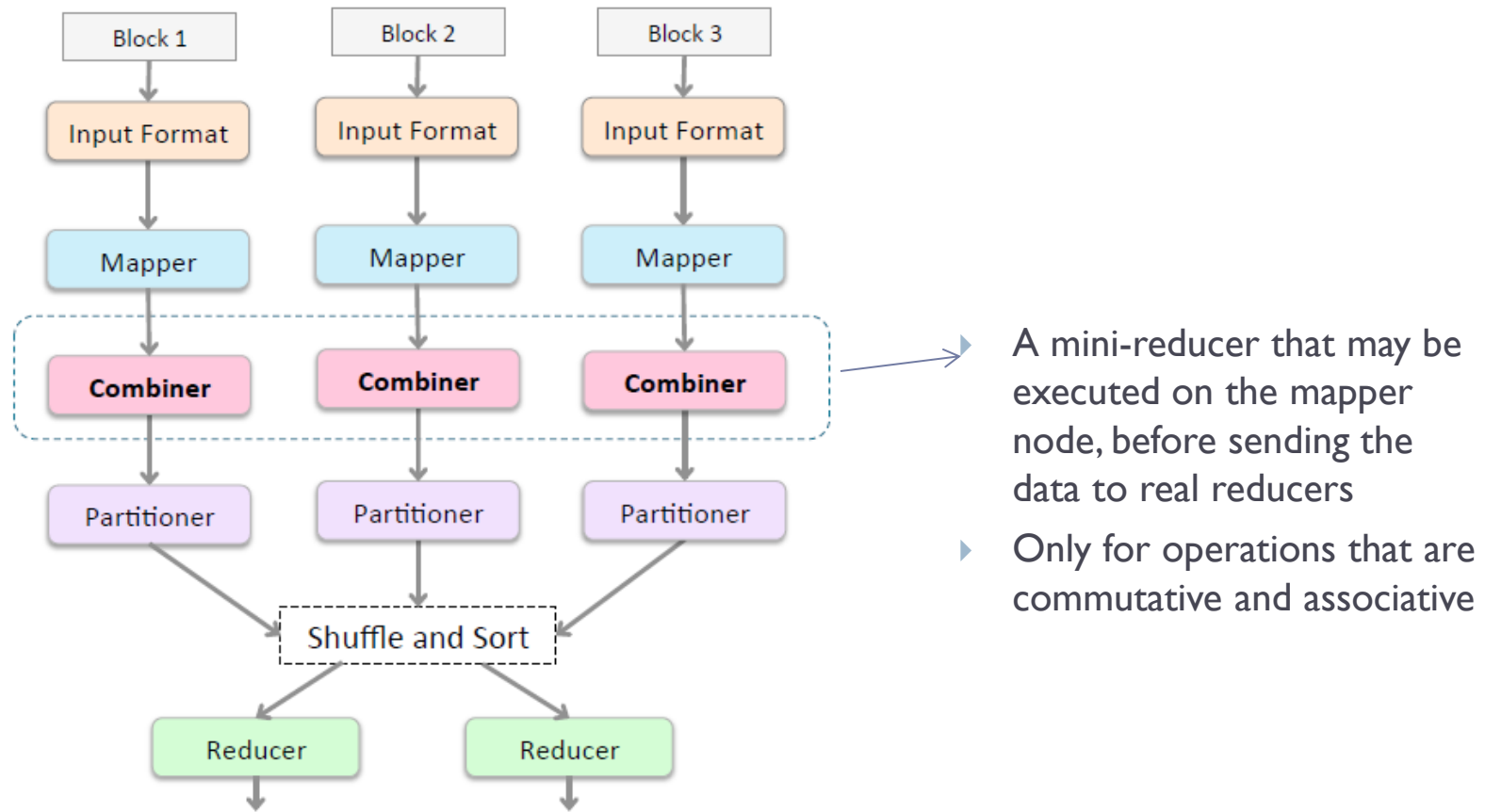
- ▶ FsShell - Hadoop wrapper program: *hadoop fs*
 - ▶ *-put*
 - ▶ *-get*
 - ▶ *-ls*
 - ▶ *-mkdir*
 - ▶ *-rm*
 - ▶ *-cat*
- ▶ Java API
- ▶ Projects dedicated to data ingestion (Hive, Pig, Spark, Sqoop, Flume, Hue)

MapReduce on Hadoop

- ▶ Automatic parallelization and execution
- ▶ Built-in fault-tolerance
- ▶ A clean abstraction for programmers
 - ▶ Hides all the “housekeeping” away from the programmer
 - ▶ Developers concentrate on writing *map* and *reduce* functions
- ▶ Monitoring tools



MapReduce on Hadoop Flow

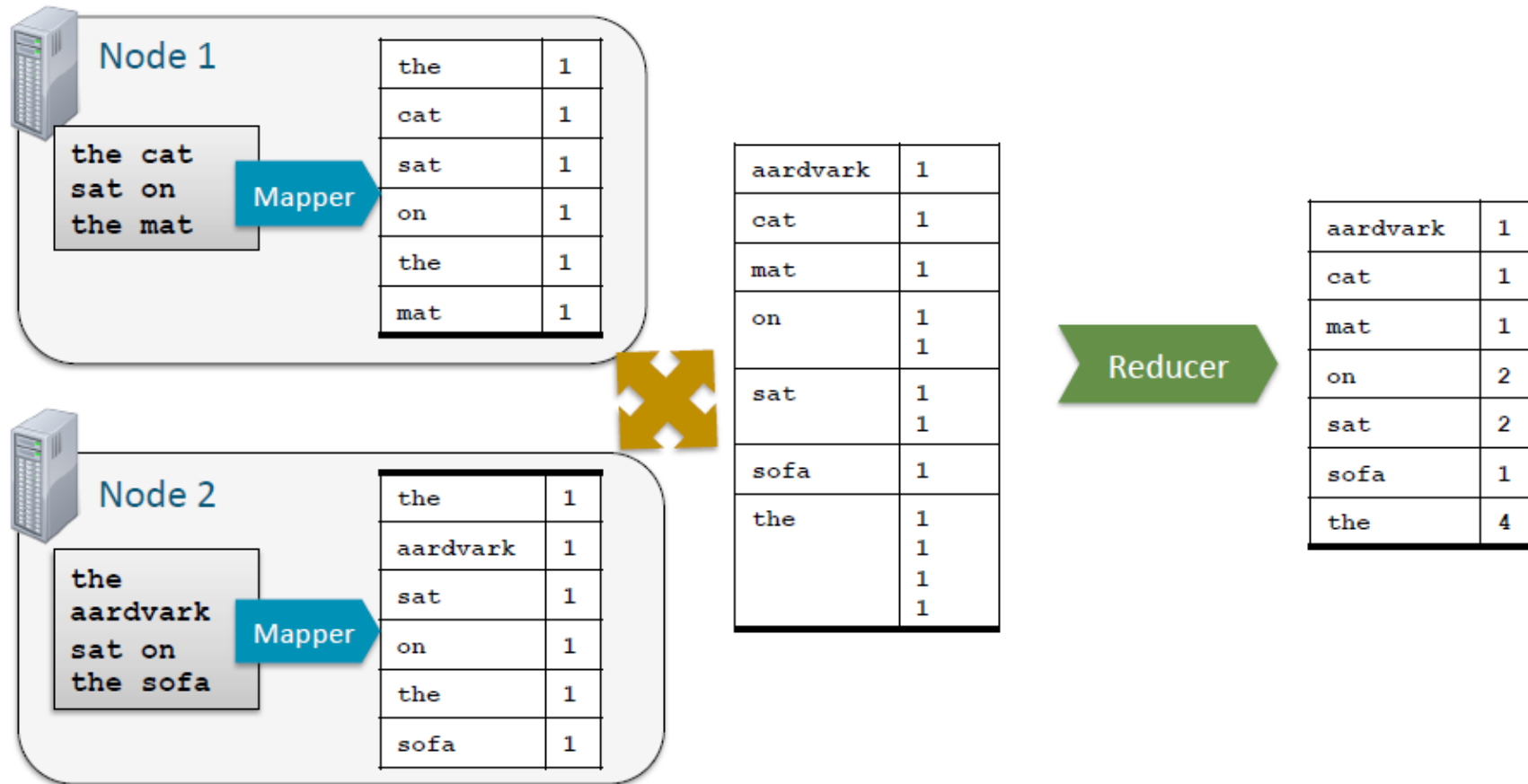


MapReduce on Hadoop

Terminology

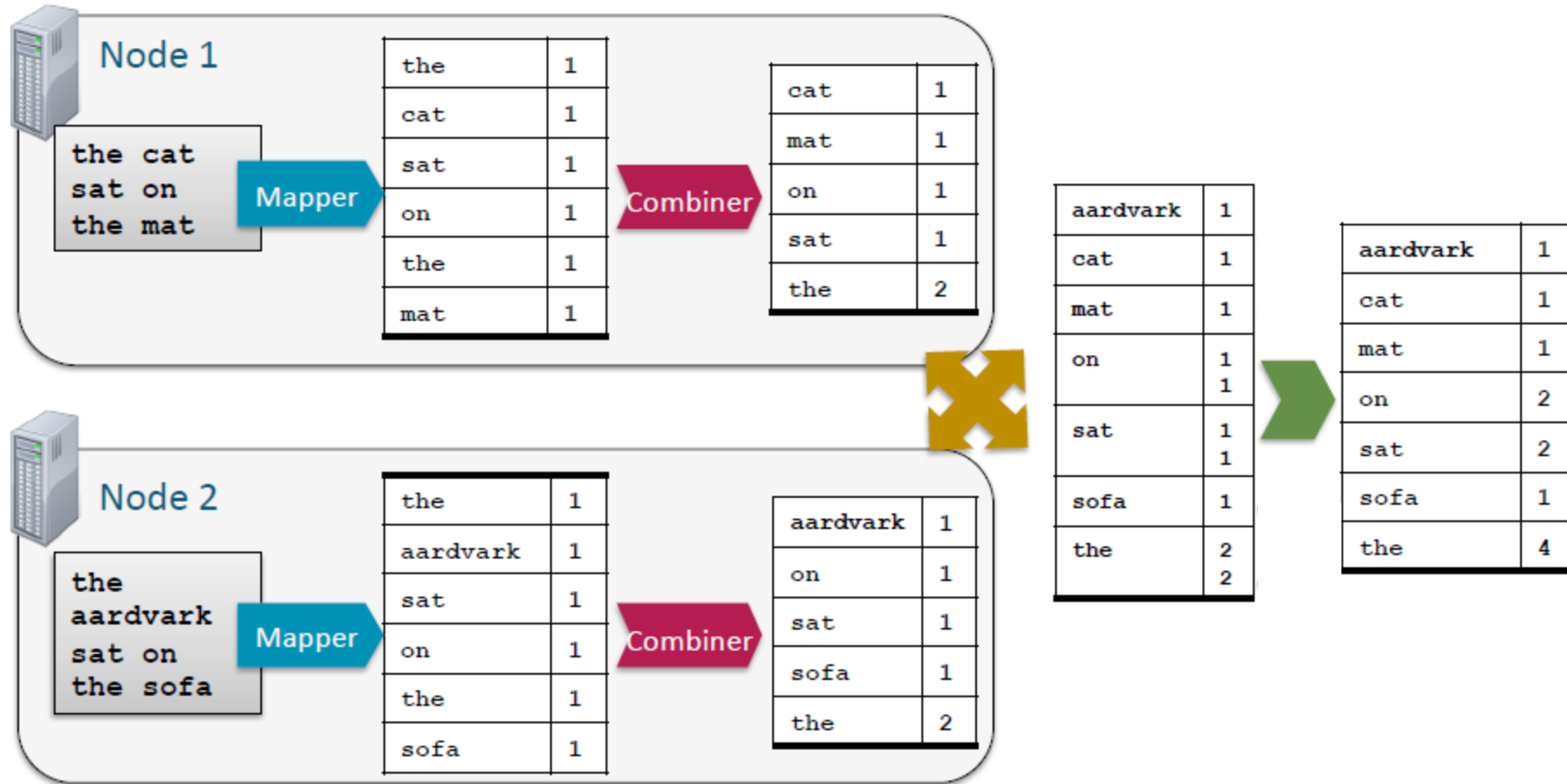
- ▶ Job – a full MapReduce application
- ▶ Task – a single Mapper or a single Reducer executed on a node
- ▶ The number of task attempts may be higher than the number of completed tasks
 - ▶ if a task attempt fails another will be started
 - ▶ Speculative execution: if a task takes longer another attempt is made on another node

Word count Map-Reduce

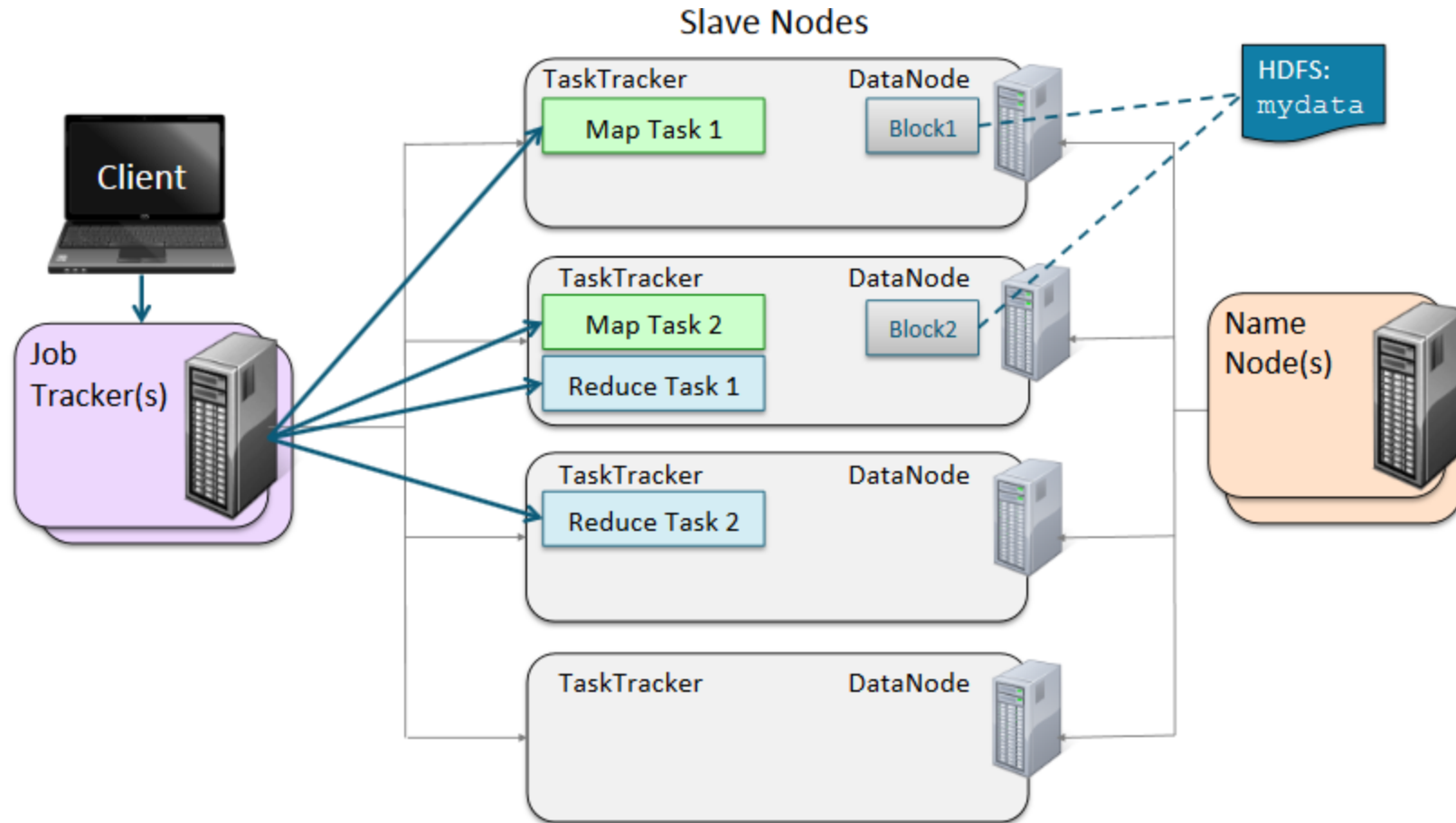


Word count

Map-(Combine)-Reduce

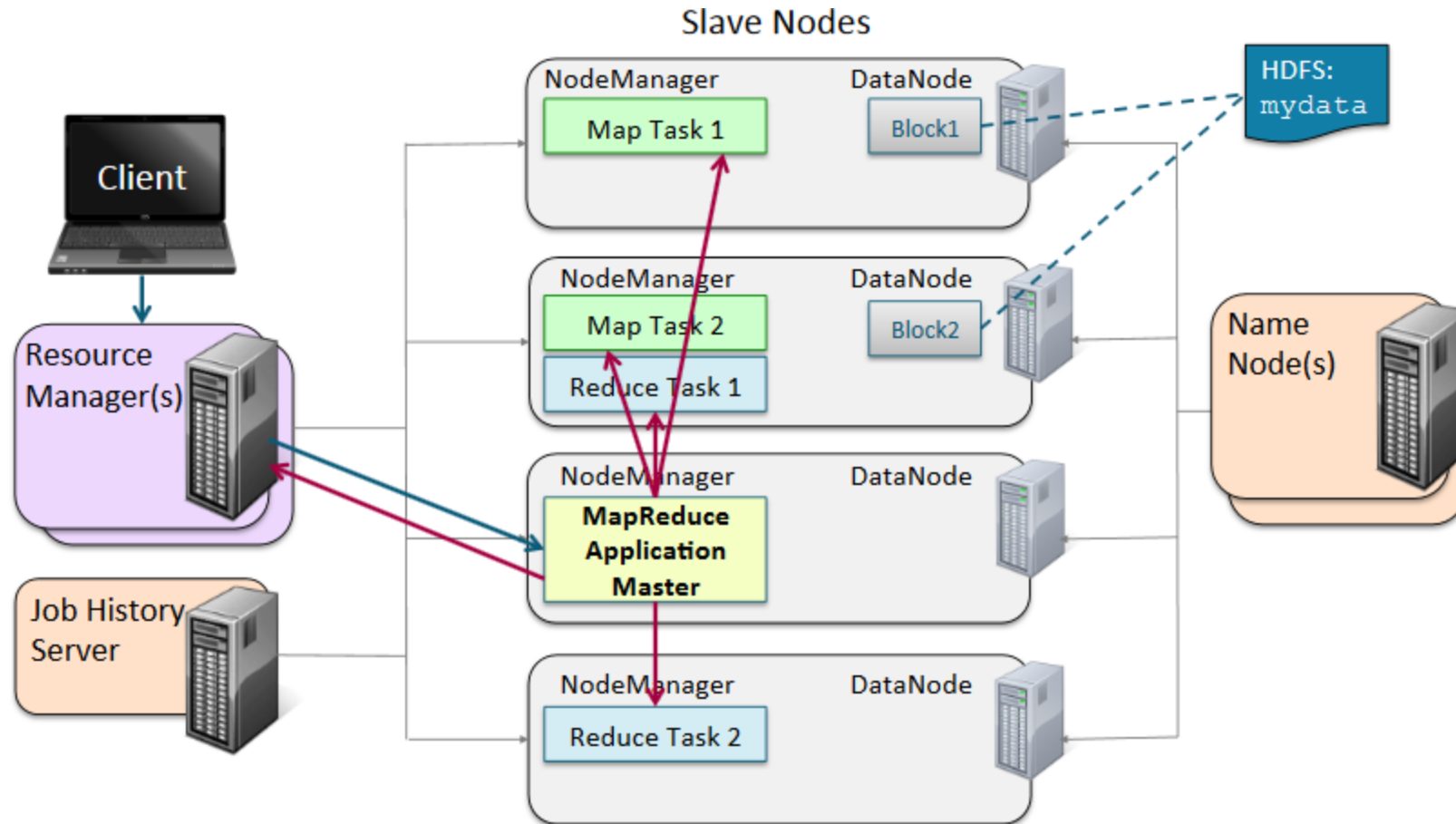


Hadoop MapReduce v1 cluster -on your virtual machine



Hadoop MapReduce v2 cluster

YARN



Map Reduce API

The Mapper

```
package solution;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString();
        for (String word : line.split("\\W+")) {
            if (word.length() > 0) {
                context.write(new Text(word), new IntWritable(1));
            }
        }
    }
}
```

Map Reduce API

The Reducer

```
package solution;

import java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int wordCount = 0;
        for (IntWritable value : values) {
            wordCount += value.get();
        }
        context.write(key, new IntWritable(wordCount));
    }
}
```

Map Reduce API

The Driver

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.printf(  
                "Usage: WordCount <input dir> <output dir>\n");  
            System.exit(-1);  
        }  
        Job job = new Job();  
        job.setJarByClass(WordCount.class);  
        job.setJobName("Word Count");  
  
        FileInputFormat.setInputPaths(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
        job.setMapperClass(WordMapper.class);  
        job.setReducerClass(SumReducer.class);  
        job.setCombinerClass(SumReducer.class);  
  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
  
        boolean success = job.waitForCompletion(true);  
        System.exit(success ? 0 : 1);  
    }  
}
```

The input and output files are in text format. When using other input/output formats, call `setInputFormatClass/` `setOutputFormatClass` methods.

The mapper's output keys and values have the same types as the reducer's output keys and values: when they are of different types, call `setMapOutputKeyClass` and `setMapOutputValueClass`

Word Count on Hadoop Execution

- ▶ `hadoop jar <jarName> solution.WordCount <input> <output>`

- ▶ Tracking the execution:

```
17/09/13 13:44:26 INFO mapred.JobClient: Running job: job_201709131338_0002
17/09/13 13:44:27 INFO mapred.JobClient: map 0% reduce 0%
17/09/13 13:44:43 INFO mapred.JobClient: map 50% reduce 0%
17/09/13 13:44:56 INFO mapred.JobClient: map 75% reduce 0%
17/09/13 13:44:57 INFO mapred.JobClient: map 100% reduce 0%
17/09/13 13:44:58 INFO mapred.JobClient: map 100% reduce 25%
17/09/13 13:45:01 INFO mapred.JobClient: map 100% reduce 100%
17/09/13 13:45:04 INFO mapred.JobClient: Job complete: job_201709131338_0002
```

- ▶ System counters at the end of the execution

File System Counters

```
FILE: Number of bytes read=1750998
FILE: Number of bytes written=36342

HDFS: Number of bytes read=5284706
HDFS: Number of bytes written=29937
```

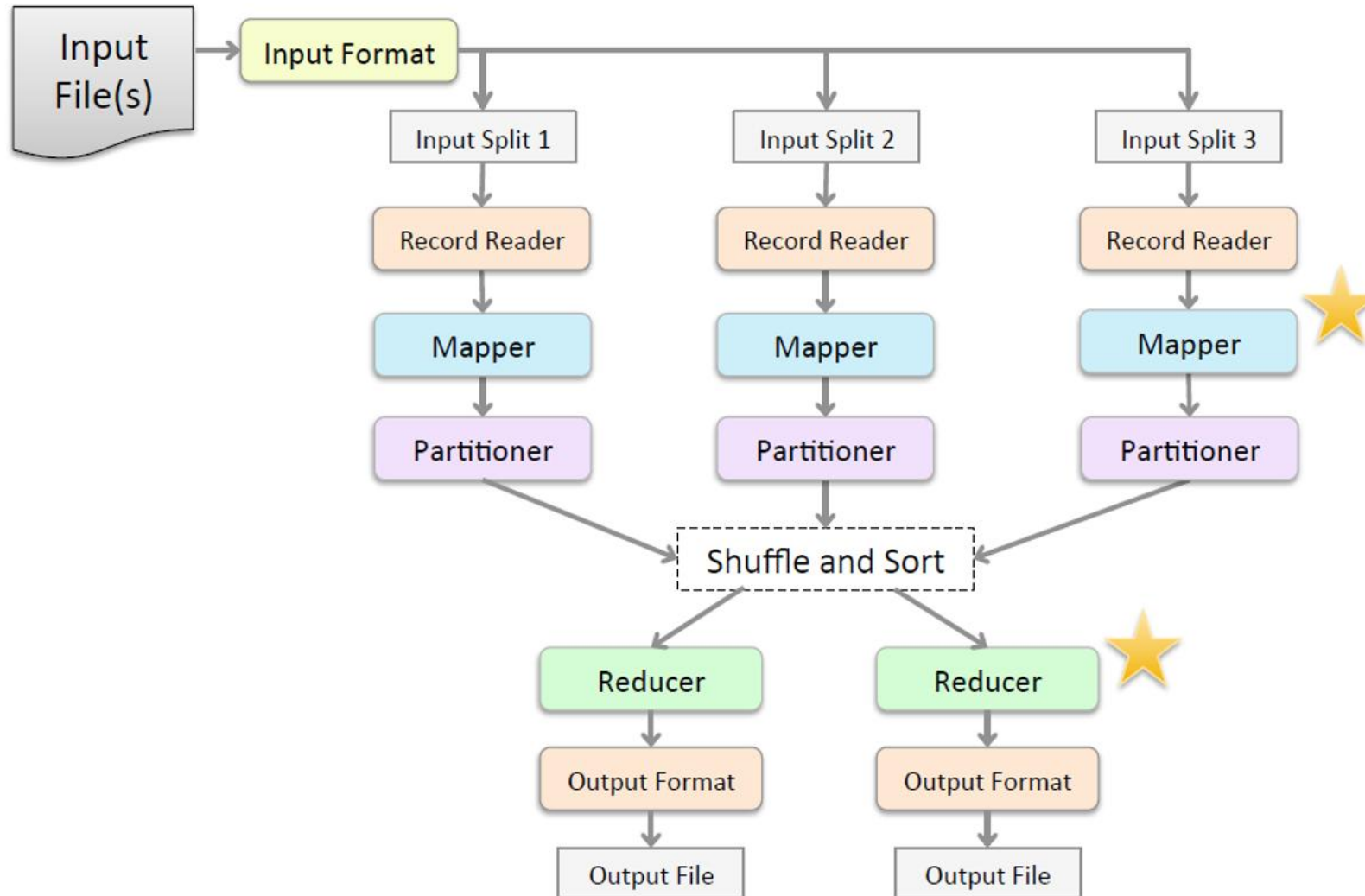
Map-Reduce Framework

```
Map input records=173126
Map output records=964453
Map output bytes=8784130
Input split bytes=475
Combine input records=964453
Combine output records=68152
Reduce input groups=29183
Reduce shuffle bytes=923137
Reduce input records=68152
Reduce output records=29183
Spilled Records=197263
CPU time spent (ms)=11180
```

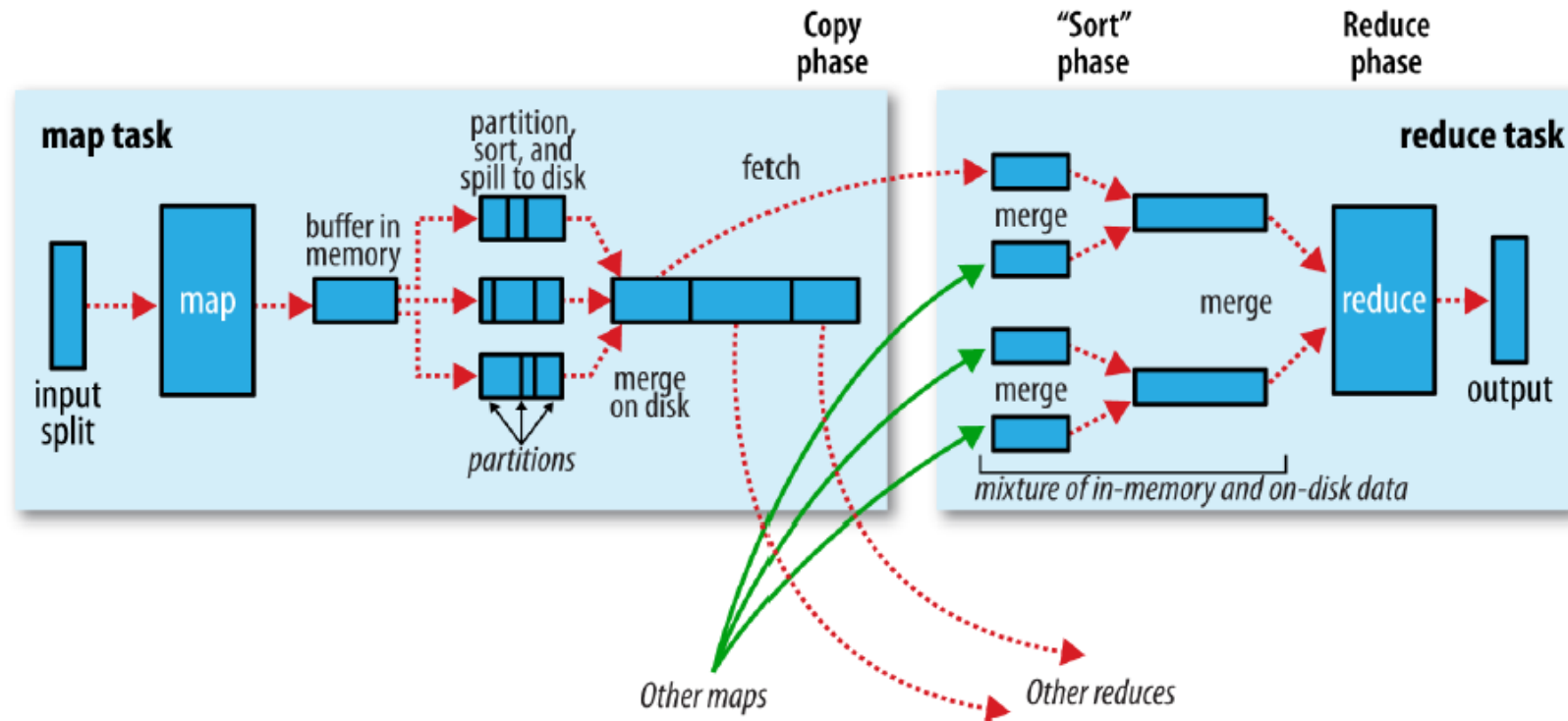
A closer look



The MapReduce flow

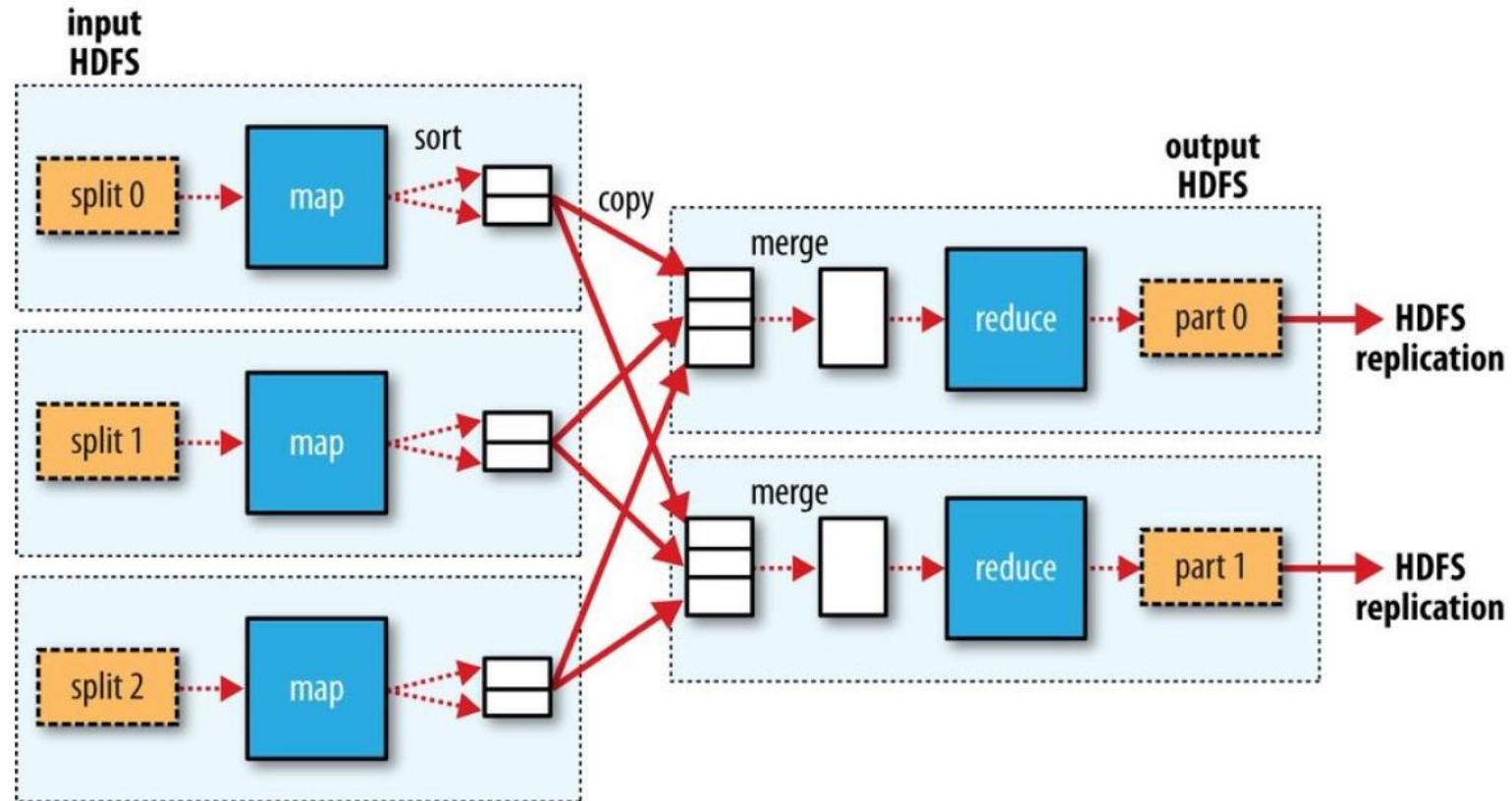


The MapReduce flow



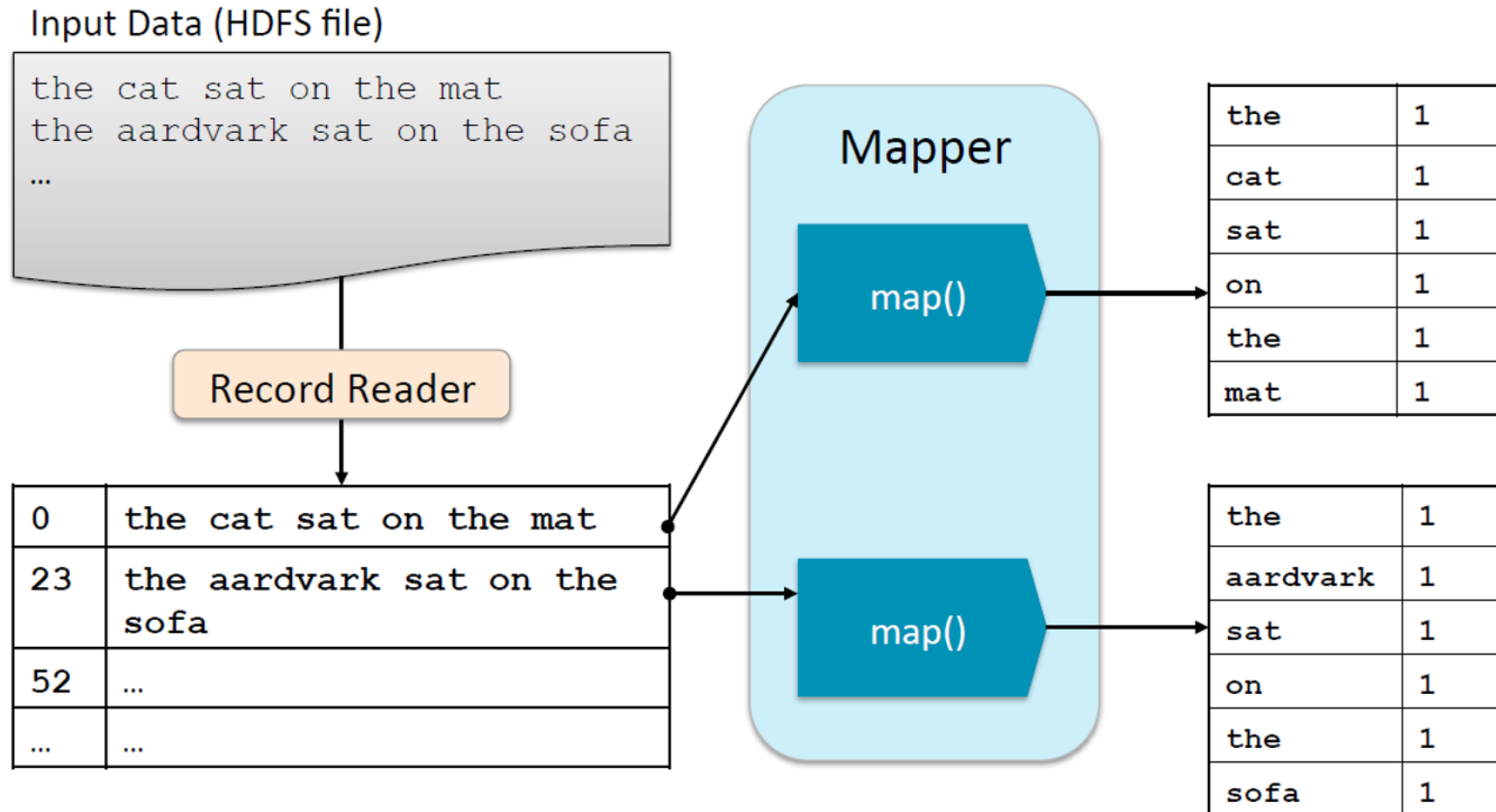
From *Hadoop: the definitive guide*

The MapReduce flow



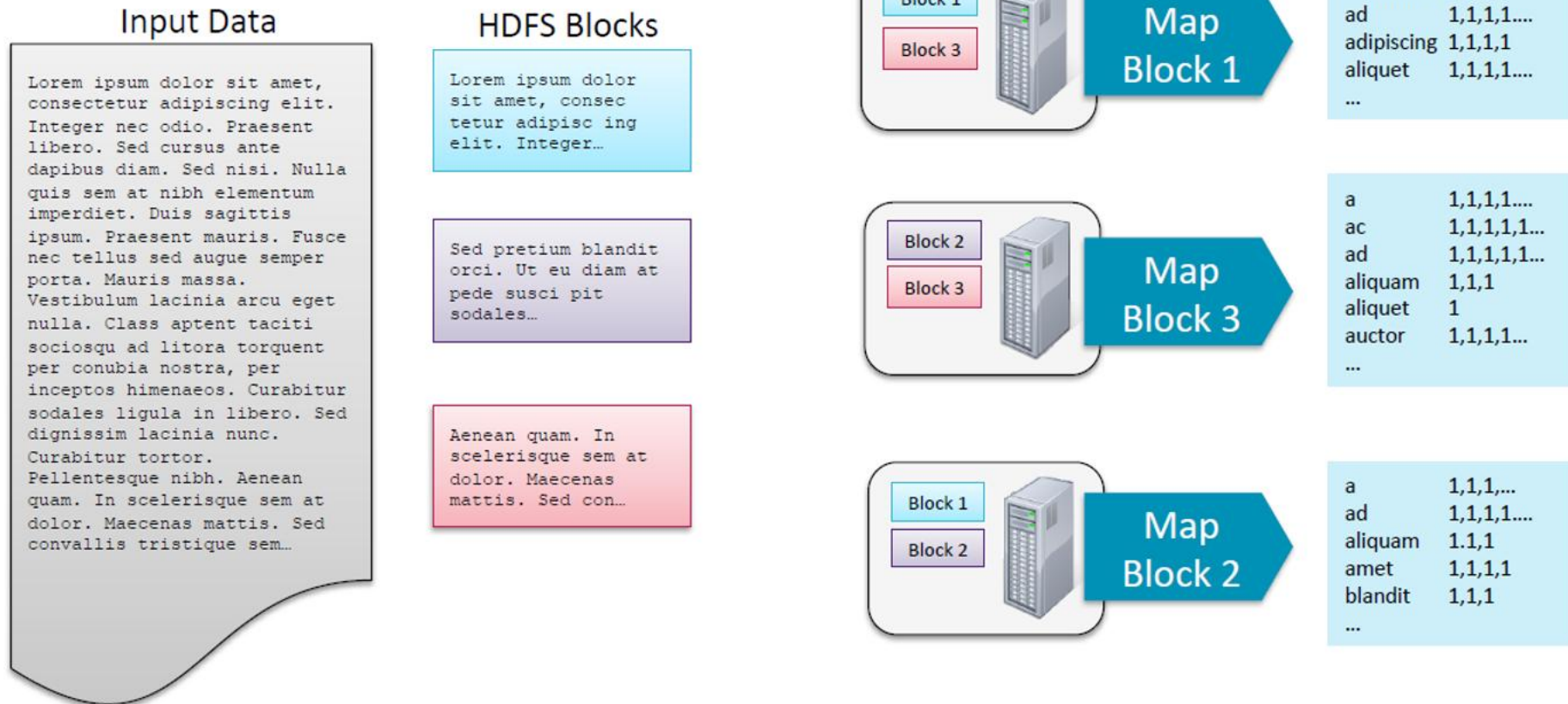
From *Hadoop: the definitive guide*

The WordCount mapper

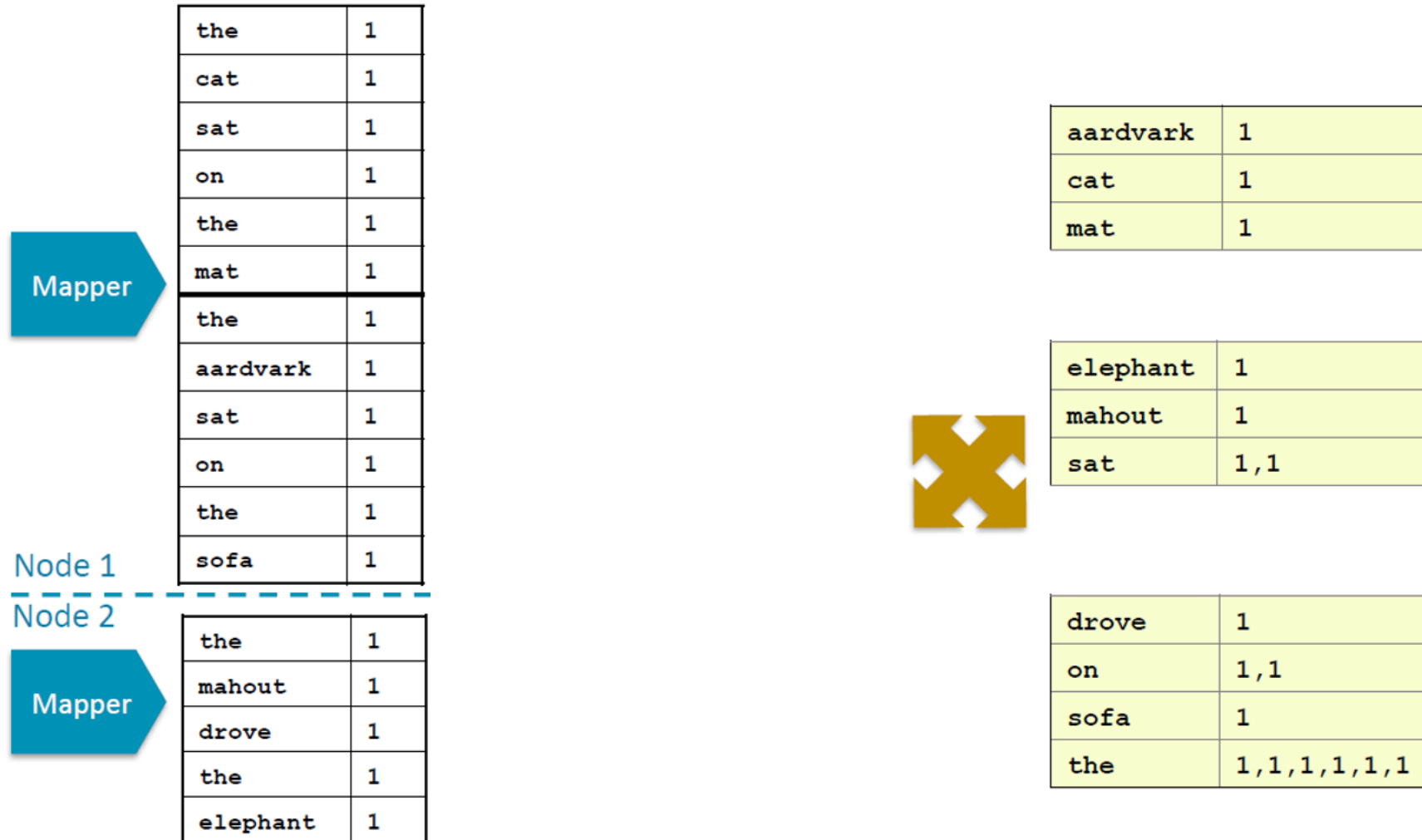


Mappers run in parallel

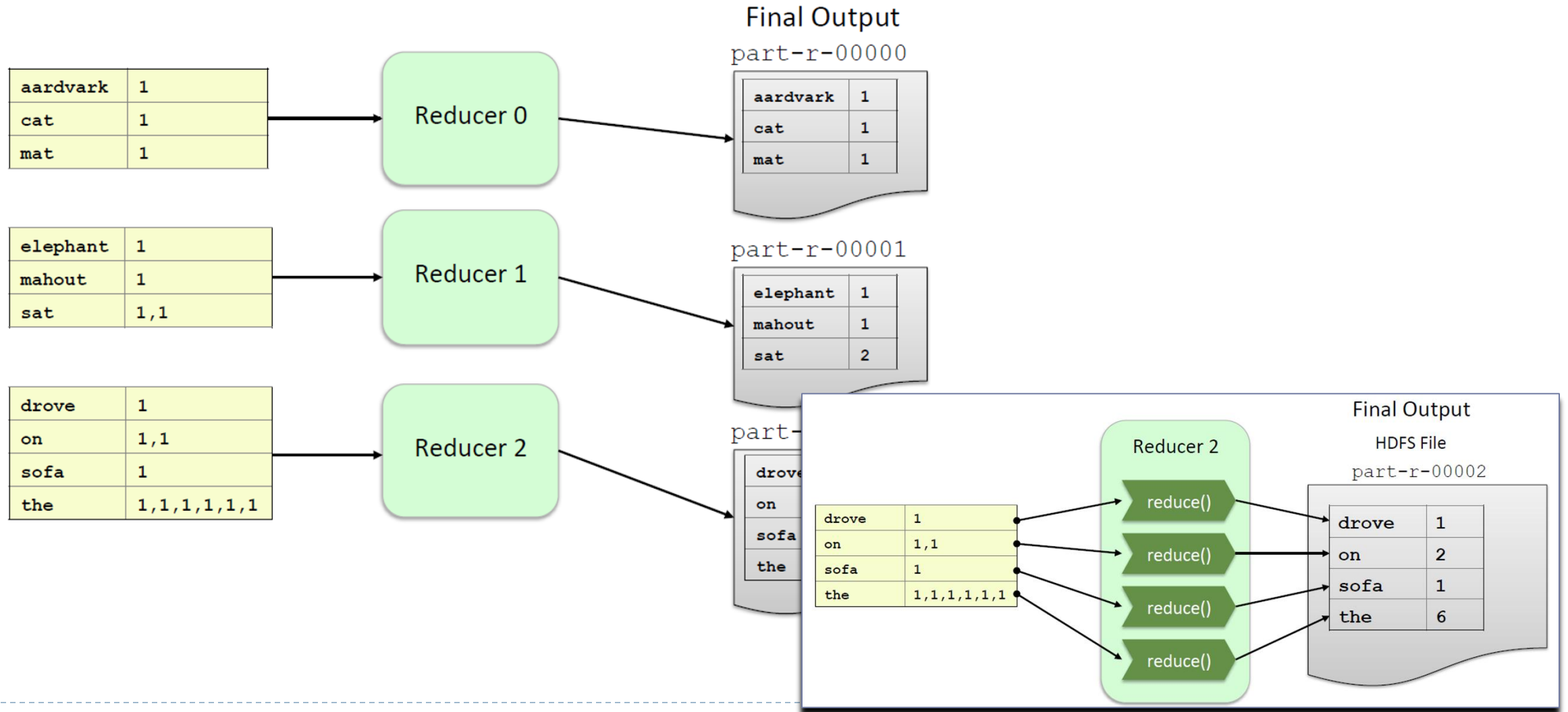
- **Hadoop runs Map tasks on the slave node where the block is stored (when possible)**
 - Many Mappers can run in parallel
 - Minimizes network traffic



WordCount: shuffle and sort



WordCount: SumReducer

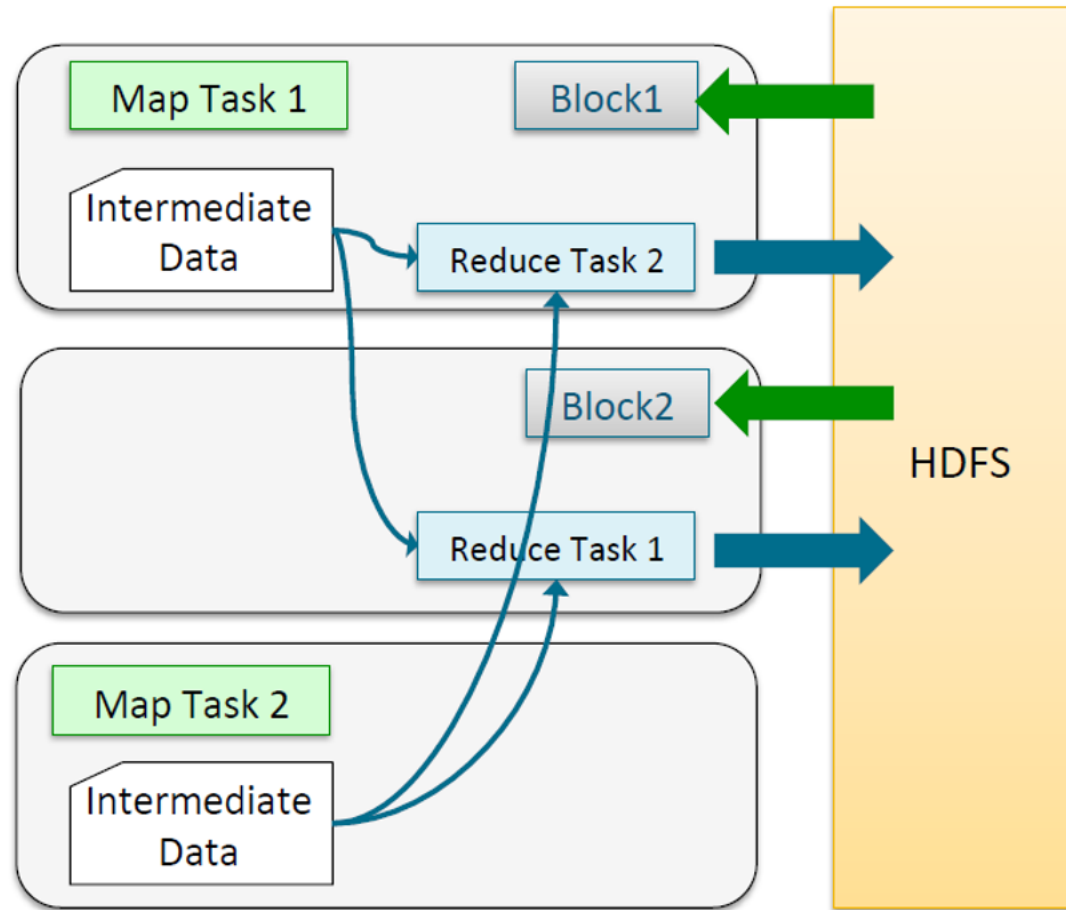


Reducers

There is no concept of data locality for Reducers

Intermediate data is transferred across the network to the Reducers

Reducers write their output to HDFS



Is Shuffle and Sort a Bottleneck?

- **It appears that the shuffle and sort phase is a bottleneck**
 - The `reduce` method in the Reducers cannot start until all Mappers have finished
- **In practice, Hadoop will start to transfer data from Mappers to Reducers as soon as the Mappers finish work**
 - This avoids a huge amount of data transfer starting as soon as the last Mapper finishes
 - The `reduce` method still does not start until all intermediate data has been transferred and sorted



Is a Slow Mapper a Bottleneck?

- **It is possible for one Map task to run more slowly than the others**
 - Perhaps due to faulty hardware, or just a very slow machine
- **It would appear that this would create a bottleneck**
 - The `reduce` method in the Reducer cannot start until every Mapper has finished
- **Hadoop uses *speculative execution* to mitigate against this**
 - If a Mapper appears to be running significantly more slowly than the others, a new instance of the Mapper will be started on another machine, operating on the same data
 - A new *task attempt* for the same task
 - The results of the first Mapper to finish will be used
 - Hadoop will kill off the Mapper which is still running



Creating and Running a MapReduce Job

- **Write the Mapper and Reducer classes**
- **Write a Driver class that configures the job and submits it to the cluster**
 - Driver classes are covered in the “Writing MapReduce” chapter

- **Compile the Mapper, Reducer, and Driver classes**

```
$ javac -classpath `hadoop classpath` MyMapper.java  
MyReducer.java MyDriver.java
```

- **Create a jar file with the Mapper, Reducer, and Driver classes**

```
$ jar cf MyMR.jar MyMapper.class MyReducer.class  
MyDriver.class
```

- **Run the `hadoop jar` command to submit the job to the Hadoop cluster**

```
$ hadoop jar MyMR.jar MyDriver in_file out_dir
```





Writing a MapReduce program



Getting Data to the Mapper

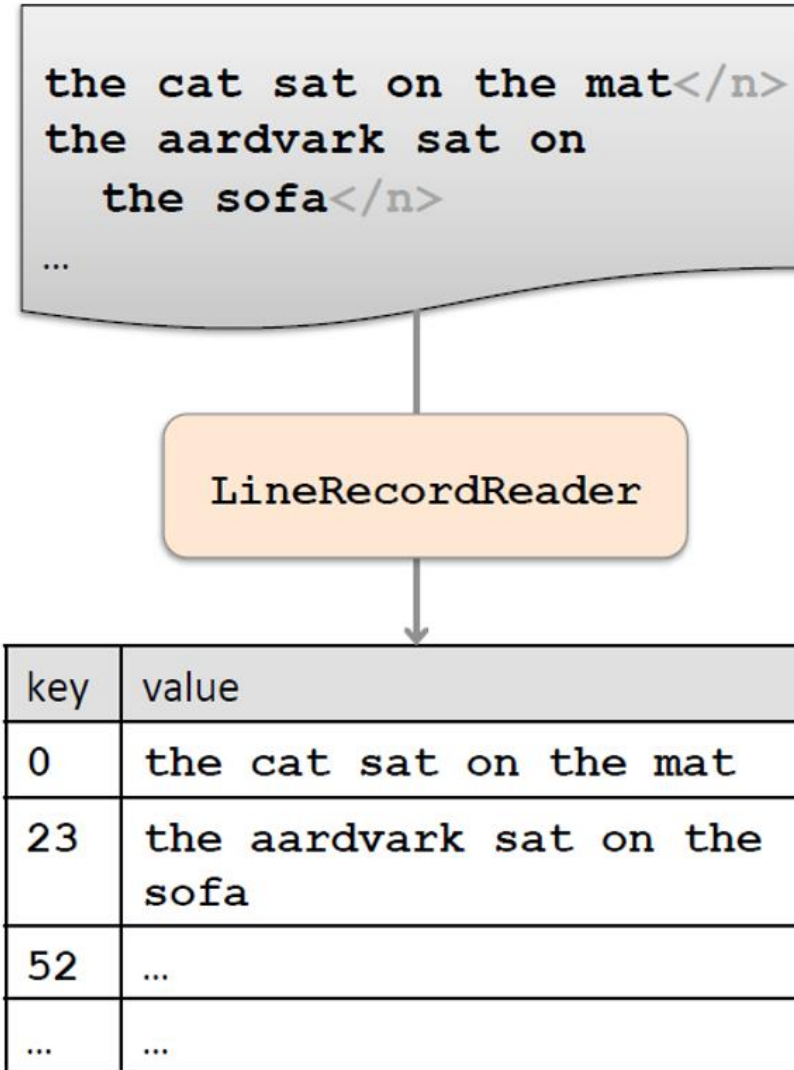
- **The data passed to the Mapper is specified by an *InputFormat***
 - Specified in the driver code
 - Defines the location of the input data
 - Typically a file or directory
 - Determines how to split the input data into *input splits*
 - Each Mapper deals with a single input split
 - Creates a `RecordReader` object
 - `RecordReader` parses the input data into key/value pairs to pass to the Mapper



Example: `TextInputFormat`

■ `TextInputFormat`

- The default
- Creates `LineRecordReader` objects
- Treats each `\n`-terminated line of a file as a value
- Key is the byte offset of that line within the file



Other Standard InputFormats

- **FileInputFormat**

- Abstract base class used for all file-based InputFormats

- **KeyValueTextInputFormat**

- Maps `\n`-terminated lines as 'key [*separator*] value'
 - By default, [*separator*] is a tab

- **SequenceFileInputFormat**

- Binary file of (key, value) pairs with some additional metadata

- **SequenceFileAsTextInputFormat**

- Similar, but maps `(key.toString(), value.toString())`



Keys and Values are Objects

- **Keys and values in Hadoop are Java Objects**
 - Not primitives
- **Values are objects which implement `Writable`**
- **Keys are objects which implement `WritableComparable`**
- **Hadoop defines its own 'box classes' for strings, integers, and so on**
 - `IntWritable` for ints
 - `LongWritable` for longs
 - `FloatWritable` for floats
 - `DoubleWritable` for doubles
 - `Text` for strings
 - Etc.



The Driver Code: Introduction

- The driver code runs on the client machine
- It configures the job, then submits it to the cluster

```
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.Job;

public class WordCount {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.out.printf("Usage: WordCount <input dir> <output dir>\n");
            System.exit(-1);
        }

        Job job = new Job();
        job.setJarByClass(WordCount.class);
        job.setJobName("Word Count");

        FileInputFormat.setInputPaths(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.setMapperClass(WordMapper.class);
        job.setReducerClass(SumReducer.class);
```

The Driver: Main Code

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        if (args.length != 2) {  
            System.out.printf("Usage: WordCount <input dir> <output dir>\n");  
            System.exit(-1);  
        }  
        Job job = new Job();  
        job.setJarByClass(WordCount.class);  
        job.setJobName("Word Count");  
  
        FileInputFormat.setInputPaths(job, new Path(args[0]));  
        FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
        job.setMapperClass(WordMapper.class);  
        job.setReducerClass(SumReducer.class);  
  
        job.setMapOutputKeyClass(Text.class);  
        job.setMapOutputValueClass(IntWritable.class);  
  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
  
        boolean success = job.waitForCompletion(true);  
        System.exit(success ? 0 : 1);  
    }  
}
```

job.setInputFormatClass(...)
job.setOutputFormatClass(...)

job.submit()

The Mapper: Complete Code

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class WordMapper extends Mapper<LongWritable, Text, Text,
IntWritable> {
    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();

        for (String word : line.split("\\W+")) {
            if (word.length() > 0) {

                context.write(new Text(word), new IntWritable(1));
            }
        }
    }
}
```

The Reducer: Complete Code

```
import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int wordCount = 0;

        for (IntWritable value : values) {
            wordCount += value.get();
        }

        context.write(key, new IntWritable(wordCount));
    }
}
```



Ensure Types Match (1)

- Mappers and Reducers declare input and output type parameters
- These must match the types used in the class

The diagram illustrates the requirement for type consistency between a Mapper's declared parameters and its implementation. A light blue box contains the Java code for the `WordMapper` class, which extends `Mapper`. The `Mapper` interface signature is `Mapper<LongWritable, Text, Text, IntWritable>`, where the first two types are input (key and value) and the last two are output (key and value). The `WordMapper` class overrides the `map` method, which takes a `LongWritable` key and a `Text` value, and writes a `Text` key and an `IntWritable` value to the context. Annotations highlight the matching types: a red box labeled 'Input key and value types' points to the `LongWritable` and `Text` parameters of the `map` method; a blue box labeled 'Output key and value types' points to the `Text` and `IntWritable` arguments of the `context.write` call. Dashed red lines connect the input types in the interface to the method parameters, and dashed blue lines connect the output types in the interface to the write arguments.

```
public class WordMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{
    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        ...
        context.write(new Text(word), new IntWritable(1));
        ...
    }
}
```

Input key and value types

Output key and value types

Ensure Types Match (2)

- Output types must also match those set in the driver

```
public class WordMapper extends Mapper<LongWritable,  
Text, Text, IntWritable> {  
    ...  
}
```

Mapper

```
public class WordCount {  
    public static void main(String[] args) throws Exception {  
        ...  
        job.setMapOutputKeyClass(Text.class);  
        job.setMapOutputValueClass(IntWritable.class);  
  
        job.setOutputKeyClass(Text.class);  
        job.setOutputValueClass(IntWritable.class);  
        ...  
    }  
}
```

driver code

```
public class SumReducer extends Reducer<Text,  
IntWritable, Text, IntWritable> {  
    ...  
}
```

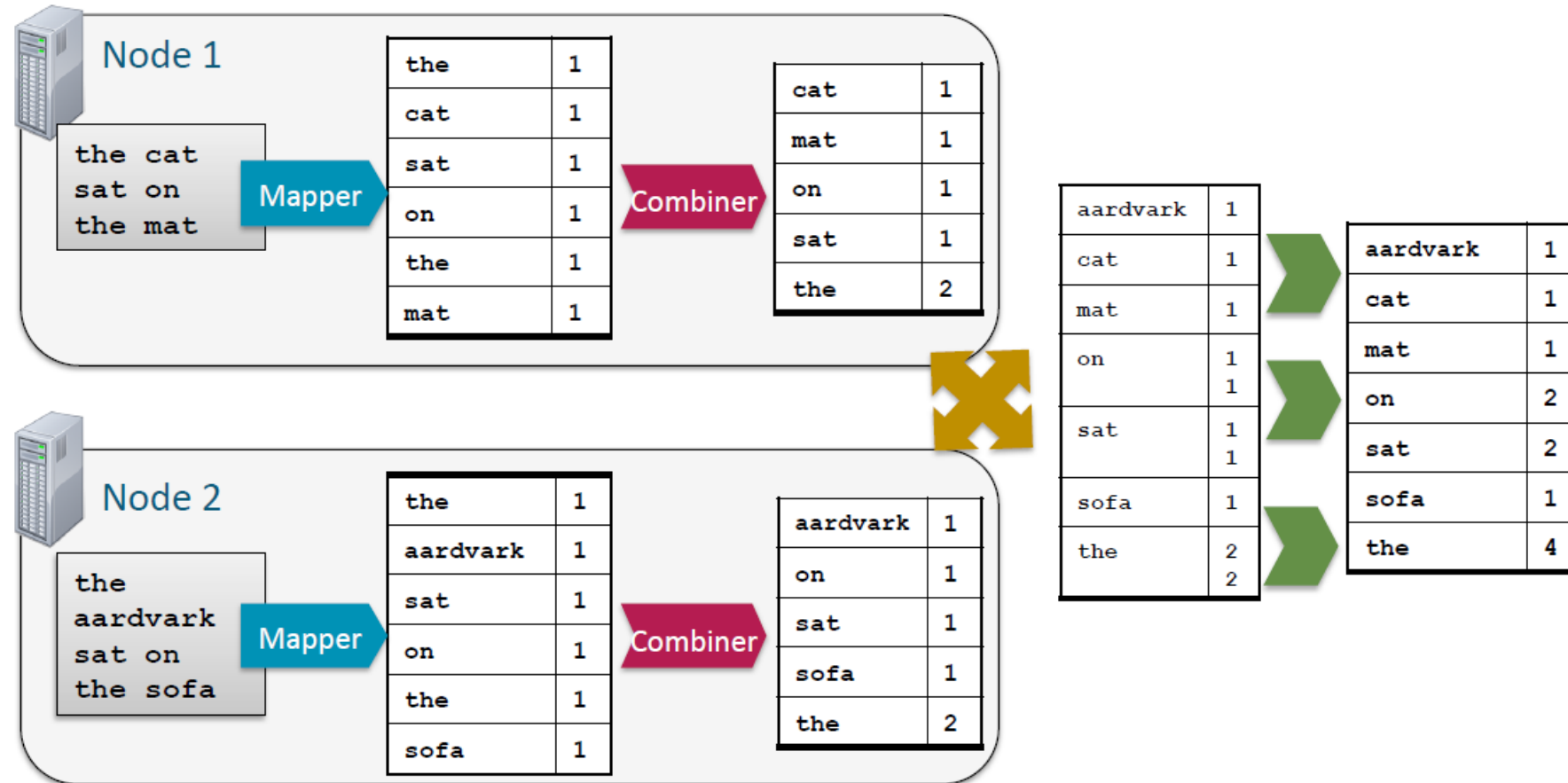
Reducer

The Combiner

- **Often, Mappers produce large amounts of intermediate data**
 - That data must be passed to the Reducers
 - This can result in a lot of network traffic
- **It is often possible to specify a *Combiner***
 - Like a 'mini-Reducer'
 - Runs locally on a single Mapper's output
 - Output from the Combiner is sent to the Reducers
- **Combiner and Reducer code are often identical**
 - Technically, this is possible if the operation performed is commutative and associative
 - Input and output data types for the Combiner/Reducer must be identical



WordCount With Combiner



Writing a Combiner

- **The Combiner uses the same signature as the Reducer**
 - Takes in a key and a list of values
 - Outputs zero or more (key, value) pairs
 - The actual method called is the **reduce** method in the class

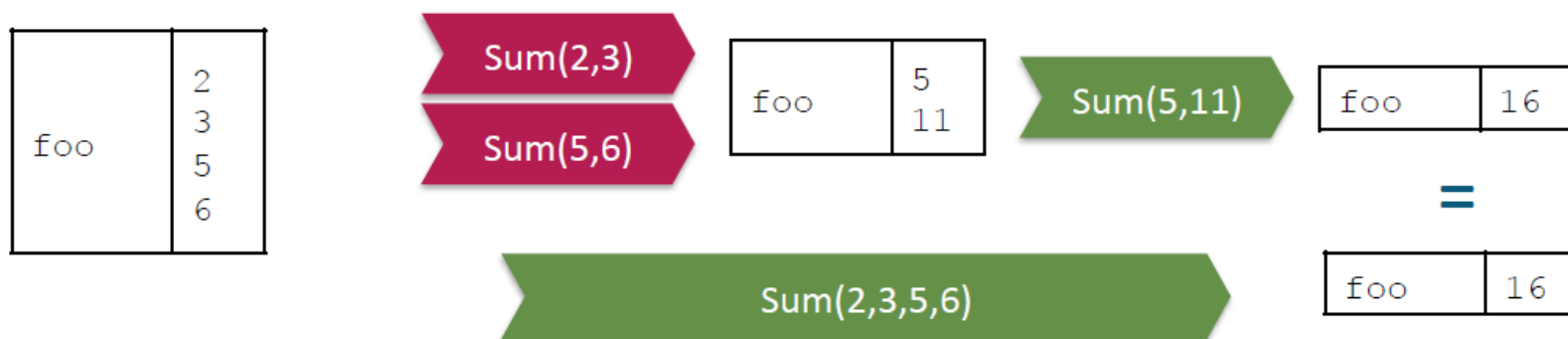
```
reduce(inter_key, [v1, v2, ...]) →  
      (result_key, result_value)
```



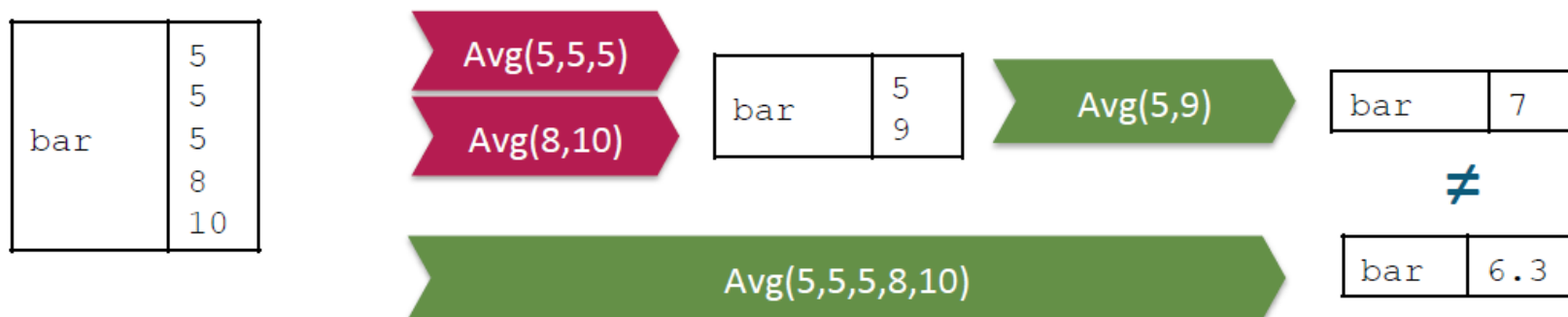
Combiners and Reducers

- Some Reducers may be used as Combiners

- If operation is associative and commutative, e.g., SumReducer



- Some Reducers *cannot* be used as a Combiner, e.g., AverageReducer



Specifying a Combiner

- **Specify the Combiner class to be used in your MapReduce code in the driver**

- Use the `setCombinerClass` method, e.g.:

```
job.setMapperClass (WordMapper.class) ;  
job.setReducerClass (SumReducer.class) ;  
job.setCombinerClass (SumReducer.class) ;
```

- **Input and output data types for the Combiner and the Reducer for a job must be identical**
- **VERY IMPORTANT: The Combiner may run once, or more than once, on the output from any given Mapper**
 - Do not put code in the Combiner which could influence your results if it runs more than once



Map-Only MapReduce Jobs

- **There are many types of job where only a Mapper is needed**
- **Examples:**
 - Image processing
 - File format conversion
 - Input data sampling
 - ETL



Creating Map-Only Jobs

- To create a Map-only job, set the number of Reducers to 0 in your Driver code

```
job.setNumReduceTasks(0);
```

- Call the `Job.setOutputKeyClass` and `Job.setOutputValueClass` methods to specify the output types
 - *Not* the `Job.setMapOutputKeyClass` and `Job.setMapOutputValueClass` methods
- Anything written using the `Context.write` method in the Mapper will be written to HDFS
 - Rather than written as intermediate data
 - One file per Mapper will be written



New API vs. Old API: Some Key Differences (1)

New API	Old API
<pre>import org.apache.hadoop.mapreduce.*</pre>	<pre>import org.apache.hadoop.mapred.*</pre>
Driver code: <pre>Configuration conf = new Configuration(); Job job = new Job(conf); job.setJarByClass(Driver.class); job.setSomeProperty(...); ... job.waitForCompletion(true);</pre>	Driver code: <pre>JobConf conf = new JobConf(Driver.class); conf.setSomeProperty(...); ... JobClient.runJob(conf);</pre>
Mapper: <pre>public class MyMapper extends Mapper { public void map(Keytype k, Valuetype v, Context c) { ... c.write(key, val); } }</pre>	Mapper: <pre>public class MyMapper extends MapReduceBase implements Mapper { public void map(Keytype k, Valuetype v, OutputCollector o, Reporter r) { ... o.collect(key, val); } }</pre>



New API vs. Old API: Some Key Differences (2)

New API	Old API
Reducer: <pre>public class MyReducer extends Reducer { public void reduce(Keytype k, Iterable<Valuetype> v, Context c) { for(Valuetype eachval : v) { // process eachval c.write(key, val); } } }</pre>	Reducer: <pre>public class MyReducer extends MapReduceBase implements Reducer { public void reduce(Keytype k, Iterator<Valuetype> v, OutputCollector o, Reporter r) { while(v.hasNext()) { // process v.next() o.collect(key, val); } } }</pre>
<code>setup(Context c)</code> (See later)	<code>configure(JobConf job)</code>
<code>cleanup(Context c)</code> (See later)	<code>close()</code>



Reusable Classes for the New API

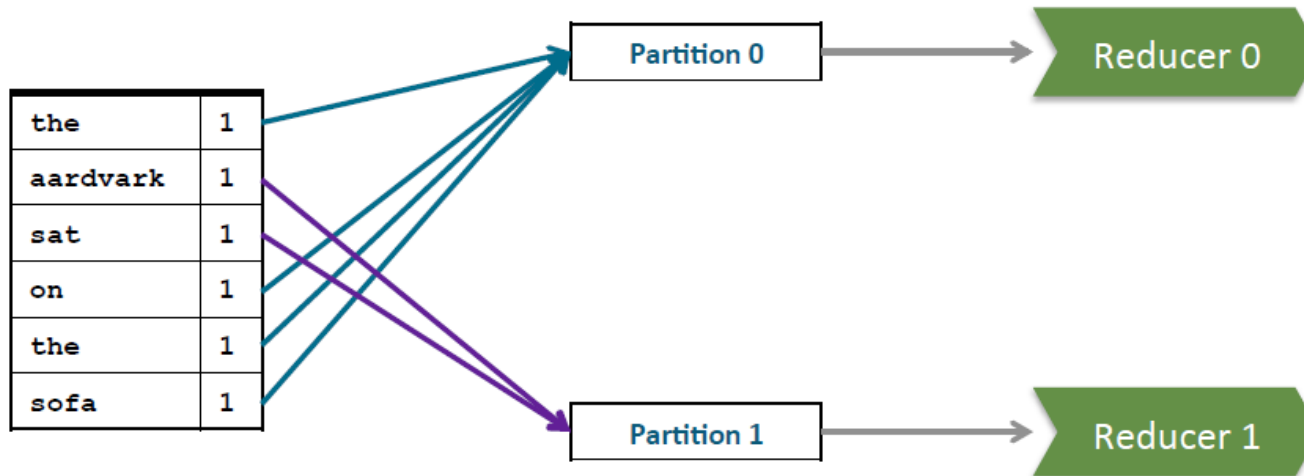
- The `org.apache.hadoop.mapreduce.lib.*/*` packages contain a library of Mappers, Reducers, and Partitioners supporting the new API
- **Example classes:**
 - `InverseMapper` – Swaps keys and values
 - `RegexMapper` – Extracts text based on a regular expression
 - `IntSumReducer`, `LongSumReducer` – Add up all values for a key
 - `TotalOrderPartitioner` – Reads a previously-created partition file and partitions based on the data from that file
 - Sample the data first to create the partition file
 - Allows you to partition your data into n partitions without hard-coding the partitioning information
- **Refer to the Javadoc for classes available in your version of CDH**
 - Available classes vary greatly from version to version



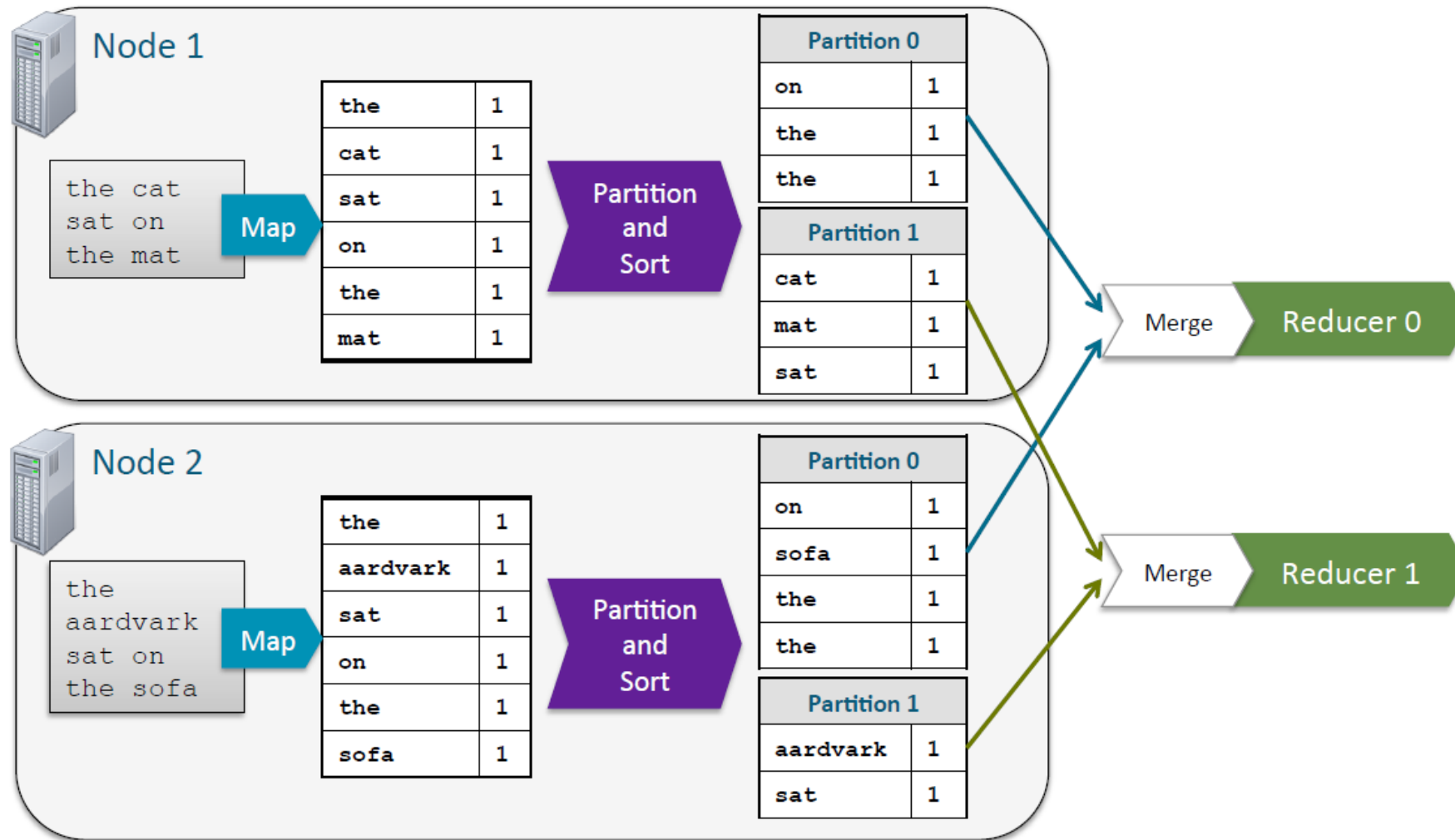
What Does the Partitioner Do?

- The Partitioner determines which Reducer each intermediate key and its associated values goes to

```
getPartion:  
  (inter_key, inter_value, num_reducers) → partition
```



Example: WordCount with Two Reducers



The Default Partitioner

- **The default Partitioner is the HashPartitioner**

- Uses the Java `hashCode` method
- Guarantees all pairs with the same key go to the same Reducer

```
public class HashPartitioner<K, V> extends Partitioner<K, V> {  
  
    public int getPartition(K key, V value, int numReduceTasks) {  
        return (key.hashCode() & Integer.MAX_VALUE) % numReduceTasks;  
    }  
}
```



How Many Reducers Do You Need?

- **An important consideration when creating your job is to determine the number of Reducers specified**
- **Default is a single Reducer**
- **With a single Reducer, one task receives *all* keys in sorted order**
 - This is sometimes advantageous if the output must be in completely sorted order
 - Can cause significant problems if there is a large amount of intermediate data
 - Node on which the Reducer is running may not have enough disk space to hold all intermediate data
 - The Reducer will take a long time to run



Jobs Which Require a Single Reducer

- **If a job needs to output a file where all keys are listed in sorted order, a single Reducer must be used**
- **Alternatively, the `TotalOrderPartitioner` can be used**
 - Uses an externally generated file which contains information about intermediate key distribution
 - Partitions data such that all keys which go to the first Reducer are smaller than any which go to the second, etc
 - In this way, multiple Reducers can be used
 - Concatenating the Reducers' output files results in a totally ordered list



Jobs Which Require a Fixed Number of Reducers

- **Some jobs will require a specific number of Reducers**
- **Example: a job must output one file per day of the week**
 - Key will be the weekday
 - Seven Reducers will be specified
 - A Partitioner will be written which sends one key to each Reducer



Jobs With a Variable Number of Reducers (1)

- **Many jobs can be run with a variable number of Reducers**
- **Developer must decide how many to specify**
 - Each Reducer should get a reasonable amount of intermediate data, but not too much
 - Chicken-and-egg problem
- **Typical way to determine how many Reducers to specify:**
 - Test the job with a relatively small test data set
 - Extrapolate to calculate the amount of intermediate data expected from the 'real' input data
 - Use that to calculate the number of Reducers which should be specified



Jobs With a Variable Number of Reducers (2)

- **Note: you should take into account the number of Reduce slots likely to be available on the cluster**
 - If your job requires one more Reduce slot than there are available, a second 'wave' of Reducers will run
 - Consisting just of that single Reducer
 - Potentially doubling the amount of time spent on the Reduce phase
 - In this case, increasing the number of Reducers further may cut down the time spent in the Reduce phase
 - Two or more waves will run, but the Reducers in each wave will have to process less data



Custom Partitioners (1)

- **Sometimes you will need to write your own Partitioner**
- **Example: your key is a custom `WritableComparable` which contains a pair of values (a, b)**
 - You may decide that all keys with the same value for **a** need to go to the same Reducer
 - The default Partitioner is not sufficient in this case
- **Custom Partitioners are also useful to avoid potential performance issues**
 - To avoid one Reducer having to deal with many very large lists of values
 - Example: in our word count job, we wouldn't want a single Reducer dealing with all the three- and four-letter words, while another only had to handle 10- and 11-letter words



Creating a Custom Partitioner

1. Create a class that extends `Partitioner`
2. Override the `getPartition` method
 - Return an int between 0 and one less than the number of Reducers
 - e.g., if there are 10 Reducers, return an int between 0 and 9

```
import org.apache.hadoop.mapreduce.Partitioner;

public class MyPartitioner<K,V> extends Partitioner<K,V> {

    @Override
    public int getPartition(K key, V value, int numReduceTasks) {
        //determine reducer number between 0 and numReduceTasks-1
        //...
        return reducer;
    }
}
```



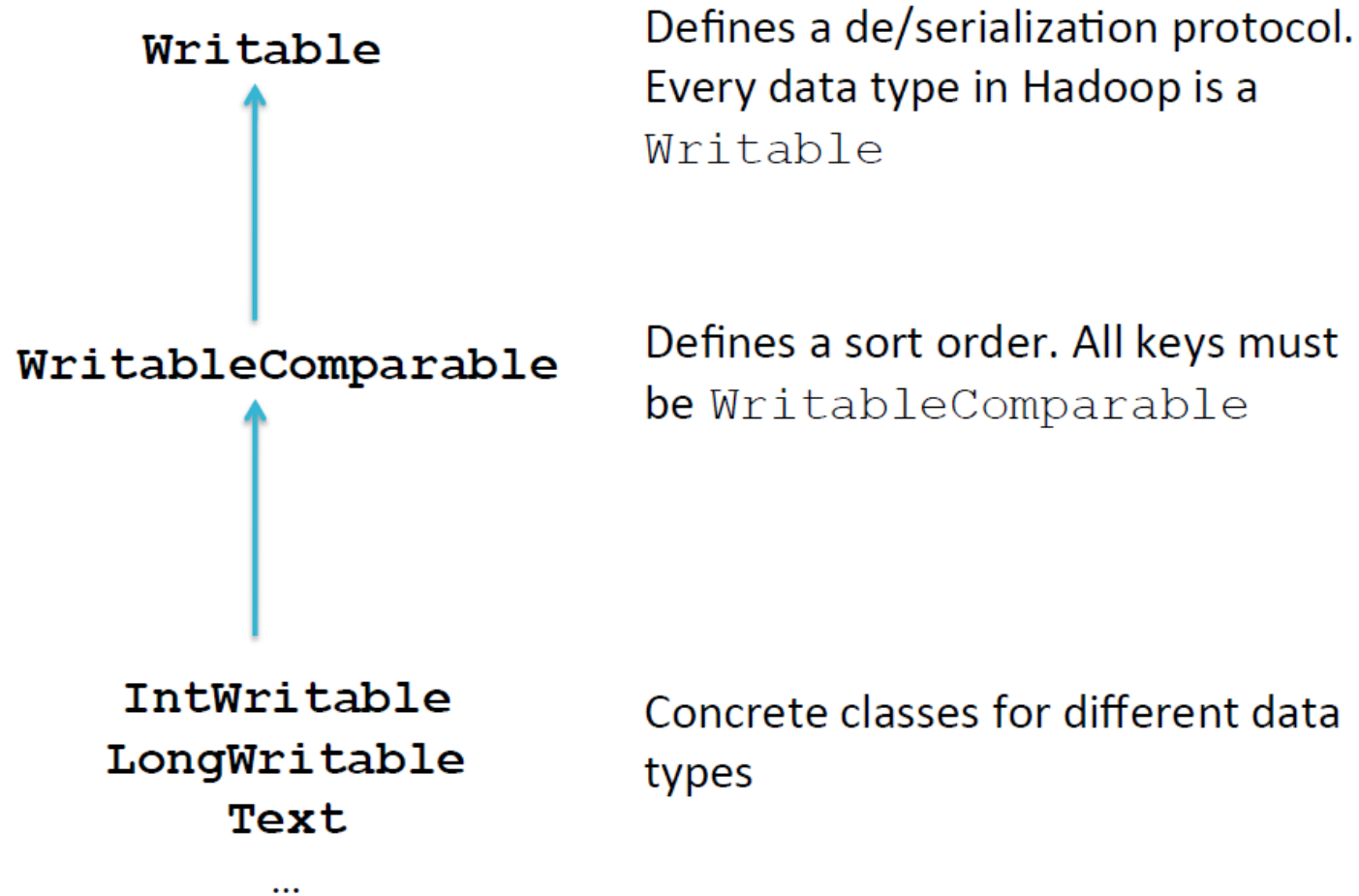
Using a Custom Partitioner

- Specify the custom Partitioner in your driver code

```
job.setPartitionerClass(MyPartitioner.class);
```



Data Types in Hadoop



'Box' Classes in Hadoop

- **Hadoop's built-in data types are 'box' classes**
 - They contain a single piece of data
 - Text: `String`
 - IntWritable: `int`
 - LongWritable: `long`
 - FloatWritable: `float`
 - etc.
- **Writable defines the wire transfer format**
 - How the data is serialized and deserialized



Creating a Complex Writable

- **Example: say we want a tuple (a, b) as the value emitted from a Mapper**
 - We could artificially construct it by, for example, saying

```
Text t = new Text(a + "," + b);  
...  
String[] arr = t.toString().split(",");
```

- **Inelegant**
- **Problematic**
 - If a or b contained commas, for example
- **Not always practical**
 - Doesn't easily work for binary objects
- **Solution: create your own Writable object**



The Writable Interface

```
public interface Writable {  
    void readFields(DataInput in);  
    void write(DataOutput out);  
}
```

- The `readFields` and `write` methods will define how your custom object will be serialized and deserialized by Hadoop
- The `DataInput` and `DataOutput` classes support
 - boolean
 - byte, char (Unicode: 2 bytes)
 - double, float, int, long,
 - String (Unicode or UTF-8)
 - Line until line terminator
 - unsigned byte, short
 - byte array



A Sample Custom Writable: DateWritable

```
class DateWritable implements Writable {
    int month, day, year;

    // Constructors omitted for brevity

    public void readFields(DataInput in) throws IOException {
        this.month = in.readInt();
        this.day = in.readInt();
        this.year = in.readInt();
    }

    public void write(DataOutput out) throws IOException {
        out.writeInt(this.month);
        out.writeInt(this.day);
        out.writeInt(this.year);
    }
}
```



WritableComparable

- **WritableComparable is a sub-interface of Writable**
 - Must implement `compareTo`, `hashCode`, `equals` methods
- **All keys in MapReduce must be WritableComparable**



Making DateWritable a WritableComparable (1)

```
class DateWritable implements WritableComparable<DateWritable> {
    int month, day, year;

    // Constructors omitted for brevity

    public void readFields (DataInput in) . . .

    public void write (DataOutput out) . . .

    public boolean equals(Object o) {
        if (o instanceof DateWritable) {
            DateWritable other = (DateWritable) o;
            return this.year == other.year && this.month == other.month
                && this.day == other.day;
        }
        return false;
    }
}
```



Making DateWritable a WritableComparable (2)

```
public int compareTo(DateWritable other) {  
    // Return -1 if this date is earlier  
    // Return 0 if dates are equal  
    // Return 1 if this date is later  
  
    if (this.year != other.year) {  
        return (this.year < other.year ? -1 : 1);  
    } else if (this.month != other.month) {  
        return (this.month < other.month ? -1 : 1);  
    } else if (this.day != other.day) {  
        return (this.day < other.day ? -1 : 1);  
    }  
    return 0;  
}  
  
public int hashCode() {  
    int seed = 163; // Arbitrary seed value  
    return this.year * seed + this.month * seed + this.day * seed;  
}  
}
```



Using Custom Types in MapReduce Jobs

- Use methods in `Job` to specify your custom key/value types
- For output of Mappers:

```
job.setMapOutputKeyClass()  
job.setMapOutputValueClass()
```

- For output of Reducers:

```
job.setOutputKeyClass()  
job.setOutputValueClass()
```



Writing MapReduce programs using streaming

The Streaming API: Motivation

- **Many organizations have developers skilled in languages other than Java, such as**
 - Ruby
 - Python
 - Perl
- **The Streaming API allows developers to use any language they wish to write Mappers and Reducers**
 - As long as the language can read from standard input and write to standard output



The Streaming API: Advantages and Disadvantages

- **Advantages of the Streaming API:**

- No need for non-Java coders to learn Java
- Fast development time
- Ability to use existing code libraries

- **Disadvantages of the Streaming API:**

- Performance
- Primarily suited for handling data that can be represented as text
- Streaming jobs can use excessive amounts of RAM or fork excessive numbers of processes
- Although Mappers and Reducers can be written using the Streaming API, Partitioners, InputFormats etc. must still be written in Java



How Streaming Works

- **To implement streaming, write separate Mapper and Reducer programs in the language(s) of your choice**
 - They will receive input via `stdin`
 - They should write their output to `stdout`
- **If `TextInputFormat` (the default) is used, the streaming Mapper just receives each line from the file on `stdin`**
 - No key is passed
- **Mapper and Reducer output should be sent to `stdout` as**
 - `key [tab] value [newline]`
- **Separators other than `tab` can be specified**



Streaming: Example Mapper

- Example streaming wordcount Mapper:

```
#!/usr/bin/env perl
while (<>) {
    chomp;
    (@words) = split /\W+/;
    foreach $w (@words) {
        print "$w\t1\n";
    }
}
```

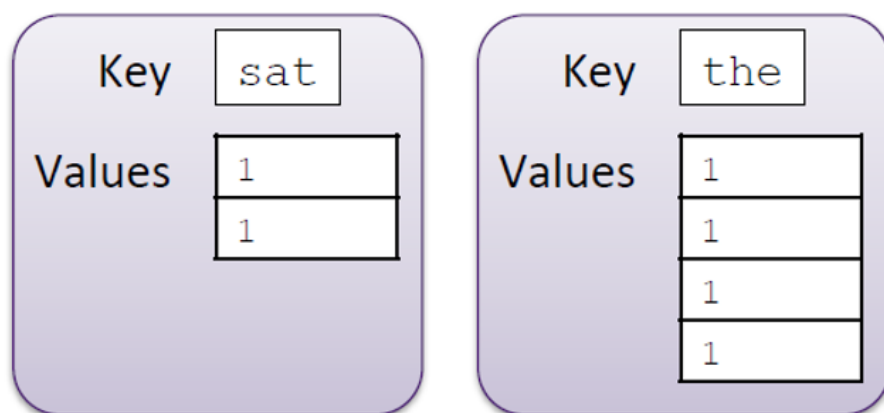
Read lines from stdin
Get rid of the trailing newline
Create an array of words
Loop through the array
Print out the key and value



Streaming Reducers: Caution

- Recall that in Java, all the values associated with a key are passed to the Reducer as an `Iterable`
- Using Hadoop Streaming, the Reducer receives its input as one key/value pair per line
- Your code will have to keep track of the key so that it can detect when values from a new key start

Java: Iterable Objects



Streaming: stdin

Diagram illustrating Streaming input for MapReduce. It shows a list of key-value pairs, one per line, where the key is the word and the value is the count '1'.

sat	1
sat	1
the	1
the	1
the	1
the	1

Streaming: Example Reducer

- Example streaming wordcount Reducer:

```
#!/usr/bin/env perl
$sum = 0;
$last = "";
while (<>) {
    ($key, $value) = split /\t/;
    $last = $key if $last eq "";
    if ($last ne $key) {
        print "$last\t$sum\n";
        $last = $key;
        $sum = 0;
    }
    $sum += $value;
}
print "$key\t$sum\n";
```

read lines from stdin
obtain the key and value
first time through
has key has changed?
if so output last key/value
start with the new key
reset sum for the new key

add value to tally sum for key

print the final pair

Launching a Streaming Job

- To launch a Streaming job, use e.g.:

```
$ hadoop jar /usr/lib/hadoop-0.20-mapreduce/contrib/\
  streaming/hadoop-streaming*.jar \  
  -input myInputDirs \  
  -output myOutputDir \  
  -mapper myMapScript.pl \  
  -reducer myReduceScript.pl \  
  -file mycode/myMapScript.pl \  
  -file mycode/myReduceScript.pl
```

- Many other command-line options are available
 - See the documentation for full details
- Note that system commands can be used as a Streaming Mapper or Reducer
 - For example: `awk`, `grep`, `sed`, or `wc`



How Hadoop does configuration

- ▶ There is a configuration API
- ▶ An instance of the `org.apache.hadoop.conf.Configuration` class – a collection of properties
 - ▶ Name (String)
 - ▶ Value (primitives: boolean, int, long, and float; other useful types such as String, Class, and `java.io.File`; collections of Strings)
- ▶ Reads the properties from xml files

```
<?xml version="1.0"?>
<configuration>
  <property>
    <name>size</name>
    <value>12</value>
  </property>

  <property>
    <name>weight</name>
    <value>light</value>
  </property>
</configuration>
```

```
Configuration conf = new Configuration();
conf.addResource("configuration-1.xml");
conf.addResource("configuration-2.xml");
```



Managing configuration

hadoop-local.xml

```
<?xml version="1.0"?>
<configuration>

  <property>
    <name>fs.defaultFS</name>
    <value>file:///</value>
  </property>

  <property>
    <name>mapreduce.framework.name</name>
    <value>local</value>
  </property>

</configuration>
```

hadoop-localhost.xml

```
<?xml version="1.0"?>
<configuration>

  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost/</value>
  </property>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>

  <property>
    <name>yarn.resourcemanager.address</name>
    <value>localhost:8032</value>
  </property>

</configuration>
```

- ▶ Conf. files by default in *etc/hadoop*
- ▶ Can set new conf. files in the driver class (previous slide)

Or at run time:

```
% hadoop fs -conf conf/hadoop-localhost.xml -ls .
Found 2 items
drwxr-xr-x  - tom supergroup          0 2014-09-08 10:19 input
drwxr-xr-x  - tom supergroup          0 2014-09-08 10:19 output
```



References

- ▶ *Hadoop: The Definitive Guide* (4th ed) by Tom White

